GenCoord: Executable Commitments for Embodied LLM Agents under Private Information

Peng He*
Tsinghua University
hepeng@tsinghua-wx.org

Junning Zhu*
Beijing Normal-Hong Kong Baptist University
t330025113@mail.bnbu.edu.cn

Haohan Yuan
University of North Carolina at Charlotte
hyuan3@charlotte.edu

Jianpeng Liang
University of California San Diego
jil652@ucsd.edu

Abstract

Embodied language-model agents must coordinate interdependent actions while each holds private task information. In dialogue-based coordination, models generate messages and interpret replies to decide what each agent should do, spending inference before the team acts. We propose GenCoord, a decentralized protocol that makes the division of work itself the message. A proposal pairs the sender's skill path with its partner's continuation in Short DSL, a compact language for expressing joint work. Compatible paths form commitments linking the sender's delivery to the partner's work after verified receipt. We check fulfillment separately from original-task success. Complete requests also let a deterministic receiver proceed without another model call. In Minecraft, trained local 0.8B agents match the task success of controlled free-form communication while reducing median commitment time by 68.2%. The same-field full-key JSON control approaches free-form output length and delay, indicating that the amount generated matters even when messages are structured. With the same prompted DeepSeek model, both protocols complete nearly all tasks, but dialogue generates 2.8 times as many output tokens. Multistep commitments improve held-out-template success from 91.3% to 98.1% with fewer decisions. These results point to a design principle for embodied cooperation: make the division of work the message, and spend inference on choosing and revising the work.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; *Natural language generation*; *Cooperation and coordination.*

Keywords

embodied multi-agent coordination, executable commitments, LLM agents, private information, Minecraft

1 Introduction

Language-model-driven embodied agents cooperate by handing over materials, processing objects, and completing actions that depend on a partner's work. We consider teams without a central planner, where each agent holds local observations and private task information that its partner needs to choose compatible work [37, 56, 60]. In dialogue-based coordination, a model generates each message and its partner processes that message before choosing an action. These exchanges consume inference on both sides, so communication can delay the agents' next coordinated action.

We illustrate this information split with two Minecraft agents fulfilling a crafting-table order. Agent A sees the order, whereas B observes which agent the task assigns to craft it. When B is assigned to craft, A should hand over planks; when A is assigned to craft, it should deliver the finished table. The product is the same, but the required actor, transfer object, and continuation differ. Such task rules represent responsibilities that can arise from authorization or resource ownership even when both agents can perform the operation. To coordinate, A needs the consequence of B's private information for their division of work, and B needs a request consistent with the order known to A.

Recent embodied systems combine language with planning and execution in several ways. CoELA chooses between communication and environmental actions, RoCo uses dialogue to coordinate robot plans, and MINDcraft supports action-by-action exchanges [40, 56, 60]. SMART-LLM constructs team plans through decomposition and allocation, while COHERENT uses a centralized task assigner with feedback from robot executors [31, 36]. Efficiency-oriented approaches reorganize communication [8, 24], reuse plans [53], or select the observations to share [3]. We instead ask what each message should contain so that decentralized agents spend fewer tokens and calls coordinating their work. That choice matters for frequent decisions and local deployment, where compact models and asynchronous execution are being developed to improve embodied responsiveness [43].

Our insight is to make the division of work itself the message. A sender proposes the skills it will perform together with the continuation it asks its partner to perform, so the transmitted object already identifies the work on both sides of the handoff. Compatible proposals form a *commitment*, which specifies what each agent owes its partner and when that work becomes due. Because a complete request gives the receiver its continuation, the receiver can check and instantiate that work deterministically. The model's role is then to interpret private information and select or revise the proposed work. We can cut model work in three ways: encode paths compactly, realize complete requests without another model call, and commit beyond the next intermediate decision. A local model can therefore spend its generation budget specifying joint work that its partner can execute directly.

In this paper, we propose *GenCoord*, which implements this idea through *Short DSL*, a compact language for skill paths and partner requests. Each agent generates a proposal from its own information,

*Equal contribution.

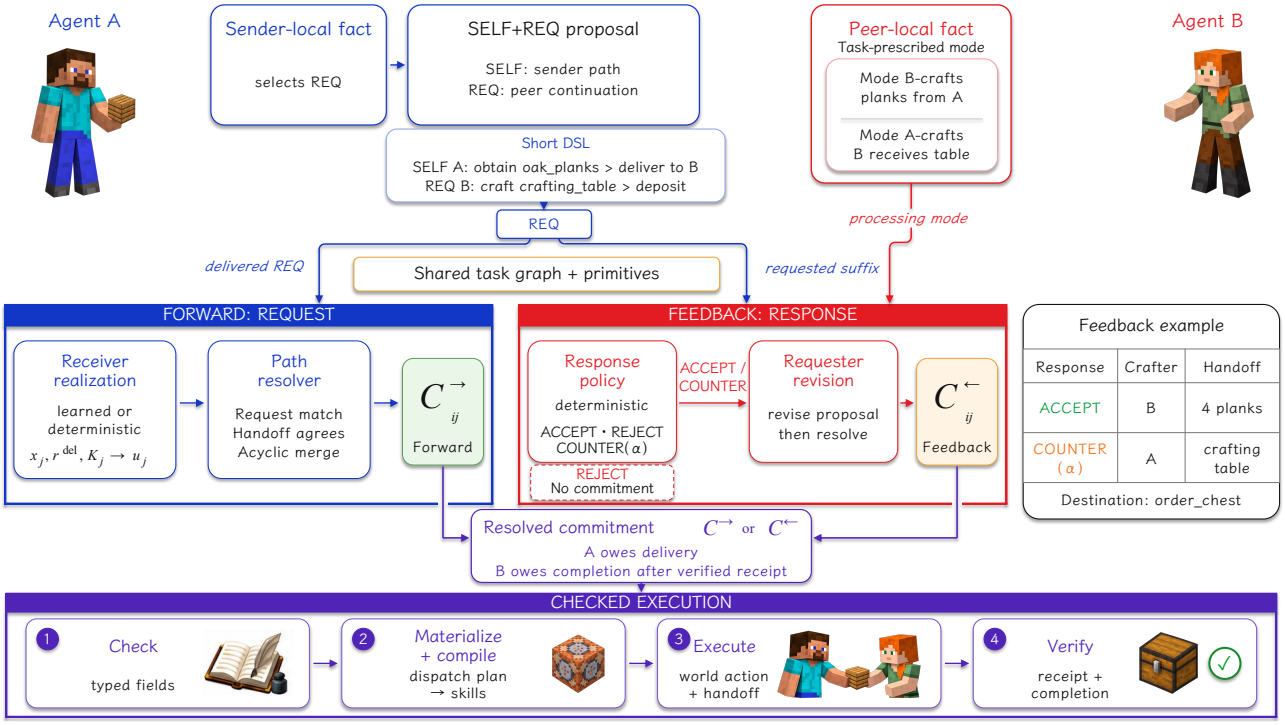


Figure 1: GenCoord in the shared-graph configuration: two information directions create executable obligations. SELF+REQ pairs the sender's work with the peer's continuation. The blue route realizes and resolves the delivered request; the red route applies a deterministic response policy, followed by requester revision and resolution. The example shows how the task-prescribed processing mode changes the handoff. The sender owes delivery, and the receiver owes completion conditional on verified receipt. Checked execution verifies fulfillment through the handoff and declared outcome.

and the protocol supports both realizing a delivered request and revising a proposal after structured feedback. Our contribution is a peer protocol in which models generate proposals for both agents' physical work and handoff dependencies, while fixed rules resolve and verify commitments. This links decentralized generation to conditional commitment semantics [10, 45, 46], with checks before dispatch and physical readback after execution (Figure 1).

We find that this protocol substantially reduces the model work needed for coordination. Trained local 0.8B agents complete the same tasks through Short DSL, JSON, and controlled free-form communication, yet Short DSL reduces peer traffic by 92.8% and median commitment time by 68.2% relative to free-form communication. Our same-field, full-key JSON control produces nearly as many tokens and takes nearly as long as free-form communication. With DeepSeek, both protocols complete nearly all tasks, while dialogue uses 8.1 times the peer traffic and 2.8 times the output tokens; Short DSL reduces mean commitment time by 30.4%. The local control isolates the encoding of identical fields, and the DeepSeek comparison establishes the cost of each complete communication procedure.

The same protocol also supports longer commitments and different model deployments. Multistep work improves held-out-template success from 91.3% to 98.1% and reduces online decisions by 32%, while structured feedback lets the requester revise the actor and handoff in response to a partner's private information. Local and hosted assignment models can use the same deterministic receiver. Our contributions are:

- **A generative coordination protocol.** GenCoord turns local skill paths and partner requests into commitments linking both agents through verified handoffs and feedback.
- **Communication as model work.** Controlled comparisons show that compact paths reduce model output, peer traffic, and commitment time, while multistep commitments improve generalization and reduce decisions.
- **Independent selection and realization.** One commitment interface supports local and hosted selectors, deterministic reception, and execution of transmitted paths.

2 Related Work

Embodied LLM agents. SayCan combines language-model proposals with skill value functions to select feasible robot actions [27]. Voyager builds reusable skills, while Code as Policies and ProgPrompt generate programs using available actions and objects [35, 44, 54]. MindCraft, MindAgent, and TeamCraft evaluate situated interaction and teamwork [1, 22, 37], while VillagerAgent and CausalMACE organize task dependencies and Parallelized Planning-Acting overlaps planning and execution [4, 16, 34]. GenCoord extends model-guided skill selection across agents by making the selected work a message for the partner.

SMART-LLM generates a multi-robot plan through decomposition, coalition formation, and allocation [31], while COHERENT couples a central task assigner to executors that report feedback [36]. Chen et al. compare centralized, decentralized, and hybrid communication [8], while Guo et al. organize embodied agents through prompted leadership [24]. GenCoord instead has each agent propose work from its own information and connects the proposals at the handoff. SmolVLA motivates local inference with compact models and responsive execution [43], while hosted models offer another operating point.

Plans and commitments. We build on accounts of shared goals and partial plans in joint intentions, SharedPlans, and STEAM [14, 23, 52], and Contract Net and allocation methods distribute work through contracts and utilities [9, 21, 49]. CLIPS infers hidden goals from actions and language, while Li et al. maintain explicit beliefs during collaboration [33, 63].

Commitment semantics specifies what agents owe one another and which events activate or discharge those obligations [20, 45, 46]. Flexible commitment protocols describe how agents enact their commitments [59], while Pippi, Kiko, and Peach address role instantiation, admissible messaging, and communication infrastructure [11–13]. Acceptance and counteroffers determine which proposed work enters that commitment [29].

Communication and computation. Learned channels, information bottlenecks, and Optima optimize agent messages [7, 19, 51, 55]. AutoGen and MetaGPT organize conversations and workflows [26, 57], while Gated Coordination, TickingCollab, μACP, and AgenticCache address escalation, timing, resource use, and reuse [30, 32, 39, 58]. MeCo retrieves and reuses plans for similar tasks to avoid invoking the model again [53]. UNCAP selects relevant vehicles and exchanges lightweight messages describing uncertain observations [3]. ReAd learns a sequential advantage function from planned trajectories to guide plan refinement, reducing interaction steps and model queries [62]. GenCoord reduces what agents generate per coordination and directly specifies work that a deterministic receiver can realize. Shared skills connect it to task grammars and ad hoc teamwork [18, 50]. We use message interventions inspired by Lowe et al. and Jaques et al. to check request and feedback semantics [28, 38], while the main comparisons measure protocol cost and multistep task performance.

GenCoord makes both agents' executable work the message, connecting their proposed skills through handoff dependencies and fixed commitment checks. Beyond Natural Language compares alternative communication formats, and OPTiMACS learns task-aware message representations for collaborative problem solving [6, 25]. In GenCoord, the transmitted content specifies the work that the runtime dispatches, so representation also determines which skills the agents undertake. BSPL, Clouseau, and Azorus address information dependencies, protocol synthesis, and commitment enactment [10, 47, 48]. We bring those concerns into a protocol whose message content consists of locally generated physical skill paths and whose commitments are checked in the environment. IntentMCTS communicates states that guide partner search [5], whereas GenCoord requests the multistep continuation that its partner will execute. CoELA, RoCo, and MINDcraft coordinate through dialogue [40, 56, 60]. GenCoord instead uses the proposed skill paths themselves to establish responsibilities, dispatch skills, and verify fulfillment.

3 Coordination under Private Information

We consider two agents $\mathcal{A} = \{a_i, a_j\}$ that cooperate through environmental actions and messages, with agent i receiving context $x_i = (o_i, h_i, m_i)$ containing its observation, execution history, and received messages. Shared primitives $\mathcal{K}_0$ define skills, argument types, and grounding rules, while $\mathcal{K}_i$ adds the task structure available to that agent. We call information in x_i or $\mathcal{K}_i$ *private* when the partner cannot observe it but needs its consequences to choose compatible work. Because private information can determine the actor, transfer object, destination, or continuation, decentralized decisions depend on what each agent observes and communicates [2].

We define a task z by its required outcome and allocation of work, using *binding* β for the selected actors, objects, and continuation. The set $\Pi^\star(z; \mathcal{K})$ contains the supported paths satisfying that task. In the workcell example, the two processing modes prescribe different crafting actors and handoff objects even though they request the same product. Our tests therefore evaluate the prescribed allocation as well as the terminal world state.

The communication requirement follows directly from the local information available at the decision. When two equiprobable tasks present the same local view but have disjoint acceptable routes, a policy choosing a route from that view has expected success at most $1/2$. We use this paired ambiguity to construct checks of message use, keeping the acceptable task routes distinct from the particular string a model emits. The design question is how to transmit the information so that a partner can act on it, which motivates the executable paths below (Supplement §S12).

4 GenCoord

GenCoord proceeds from a proposed division of work to a resolved commitment, then checks fulfillment against the execution trace. Its key design choice is to generate the work that each party undertakes in the same representation that the receiver and runtime inspect.

4.1 From Paths to Commitments

A grounded task $\tau = (a, k, \mathbf{v}, \ell, P)$ specifies an actor, skill, typed arguments, location, and predecessors. A proposal $q_i = (s_i, r_{i\to j})$ pairs the sender's path with a requested continuation for the peer, so both sides of the intended handoff are explicit. We serialize these paths in Short DSL as `SELF` and `REQ`, using `>` for precedence, as in this workcell proposal:

```
SELF resource.obtain(q=4,item=oak_planks)
  > resource.deliver(q=4,item=oak_planks,to=agent_b)
REQ agent_b craft.item(q=1,input=oak_planks,
  item=crafting_table)
  > resource.deliver(q=1,item=crafting_table,
    dst=order_chest)
```

When the paths agree on the handoff and their dependencies are acyclic, we merge them into $C_{ij} = (S_i, S_j, \beta, P_C)$, which specifies each agent's work and its ordering. A commitment is created at this resolution step, after the proposed assignment has passed the interaction checks and both sides specify compatible work. The resolver checks compatibility between choices made by the agents. For example, a plank delivery can support a receiver's crafting step

only when the actors, item quantity, and dependency agree. This compatibility is established symbolically before dispatch, while the later receipt check establishes that the promised material was actually transferred. Keeping those events distinct allows the same proposal representation to support both generation and deterministic interpretation.

Conditional commitments explain why receipt matters to the receiver's responsibility: the sender's promised delivery establishes the precondition under which the receiver owes its remaining work. Writing $C(d, c, p, q)$ for debtor d owing creditor c outcome q when p holds [46], we assign

$$\mathrm{Obl}(C_{ij}) = \{C(a_i, a_j, \top, H_\beta),\ C(a_j, a_i, H_\beta, T_\beta)\}. \tag{1}$$

Here H_β is verified receipt of the handoff and T_β is the declared terminal outcome. Receipt fulfills the sender's obligation and activates the receiver's remaining work, whose completion is checked in the world [10]. The receipt remains valid when crafting consumes the transferred materials. In the example, the receiver owes the requested crafting and deposit after it has received the planks, while the sender is responsible for making that transfer. We record the transfer as an event in the trace, so subsequent consumption preserves evidence that the sender fulfilled its delivery.

We distinguish that fulfillment from the original task's success because a commitment can specify the wrong assignment. For a trace ξ ending in world state ω_T,

$$F(C_{ij}, \xi) = H_\beta(\xi) \wedge T_\beta(\xi),$$
$$G_z(\xi) = \mathrm{ExecOK}(\xi) \wedge P_z(\omega_T) \wedge A_z(\xi).$$

The predicates P_z and A_z specify the original outcome and allocation, respectively. Thus F evaluates the committed work, whereas G_z evaluates whether that work satisfied the task.

4.2 Requests and Feedback

In a forward interaction, the receiver produces $u_j = R_j(x_j, r^{\mathrm{del}}_{i\to j}, \mathcal{K}_j)$ from its local context, executable prior, and delivered request, using the inventory expected after the proposed handoff to generate or deterministically instantiate the continuation. The path resolver forms $C^{\rightarrow}_{ij} = \mathrm{Resolve}_{\rightarrow}((s_i, r^{\mathrm{del}}_{i\to j}), u_j)$ when the continuation matches the normalized delivered request, the handoff fields agree, and the merged dependencies are acyclic. Using the delivered request also gives message substitutions a precise operational meaning.

Feedback instead lets the requester revise who performs the work. A deterministic policy reads the requested suffix and the peer's processing mode, returning Accept, Reject, or Counter(α) for an admissible alternative. Acceptance preserves the proposed branch, while a counteroffer changes the allocation and preserves the goal and peer. The requester then generates a revised proposal from its context, initial proposal, and response, after which the feedback resolver checks the resulting handoff and path. Rejection creates no commitment, whereas a counteroffer in our example asks the requester to craft before handing over the finished table. The response therefore communicates the consequence of the peer's mode in the form of a concrete alternative that the requester can incorporate into its own path. The resolver checks the revised paths against the response, including who processes the material and what the requester will deliver. These two interactions place the additional model call on the agent that still has work to determine: the peer realizes the incoming request in the forward route, whereas the original requester revises its proposal in the feedback route. A deterministic peer can perform the former operation when the request already specifies its full continuation.

4.3 Realization and Execution

We parse and check the generated message to validate identities, skills, arguments, destinations, and dependencies, after which the materializer constructs the task actions needed for dispatch. The formation backend chooses and expresses the assignment, while the materializer supplies the task details available through the shared graph or received continuation. In the shared-graph configuration it expands an admitted binding through the available task graph. When the sender keeps its graph private, the receiver instead derives this work from the delivered continuation and shared primitives.

After the compiler converts the dispatch plan to Mineflayer skills with navigation and execution parameters [41], we execute it and inspect inventories and world objects to verify the transfer and declared outcome (Figure 2). Because formation, fulfillment, and task success are distinct checks, we can locate whether an unsuccessful trial failed to form a commitment, failed to carry it out, or carried out an inappropriate commitment. Complete paths consequently support deterministic realization without another model call, since the receiver's remaining task is to validate and instantiate the continuation it has already received. For a generated continuation, the same checks determine whether the model followed the request, so the execution interface remains stable across both implementations.

4.4 Choosing the Implementation

Direct Short DSL (Direct) uses Qwen3.5-0.8B to generate complete paths, while *Factor-Code + Rule* (Factor) predicts a short binding code that a deterministic composer expands into the same object [42]. A rule can transfer an explicitly supplied binding, and lexical or embedding retrieval can select one from a description. Hosted generation provides another choice when greater assignment accuracy is valuable, although the service's response time becomes part of the agent's decision latency. Each backend expresses its selected assignment through the same receiver-facing request interface.

Receiver knowledge provides a second design choice. A shared directory maps codes to known continuations, whereas `PATH` transmits an ordered continuation and `SLICE` transmits the relevant graph fragment. `GOAL` instead supplies a target and constraints, which an adapter realizes by searching inventory, container, and block states from the specified handoff. Because this adapter checks the target and constraints, a different legal path can satisfy the same commitment. The sender can transmit a complete path or leave the receiver to recover steps from shared knowledge or search, trading message length against receiver computation (Supplement §S7–S8).

5 Experimental Setup

5.1 Tasks and Inputs

We evaluate Minecraft cooperation over eight templates in four families: Destination, Recipe, Allocation, and Active Branch (Figure 3). Each template pairs two task assignments with balanced frequencies and swaps sender and receiver identities across two role

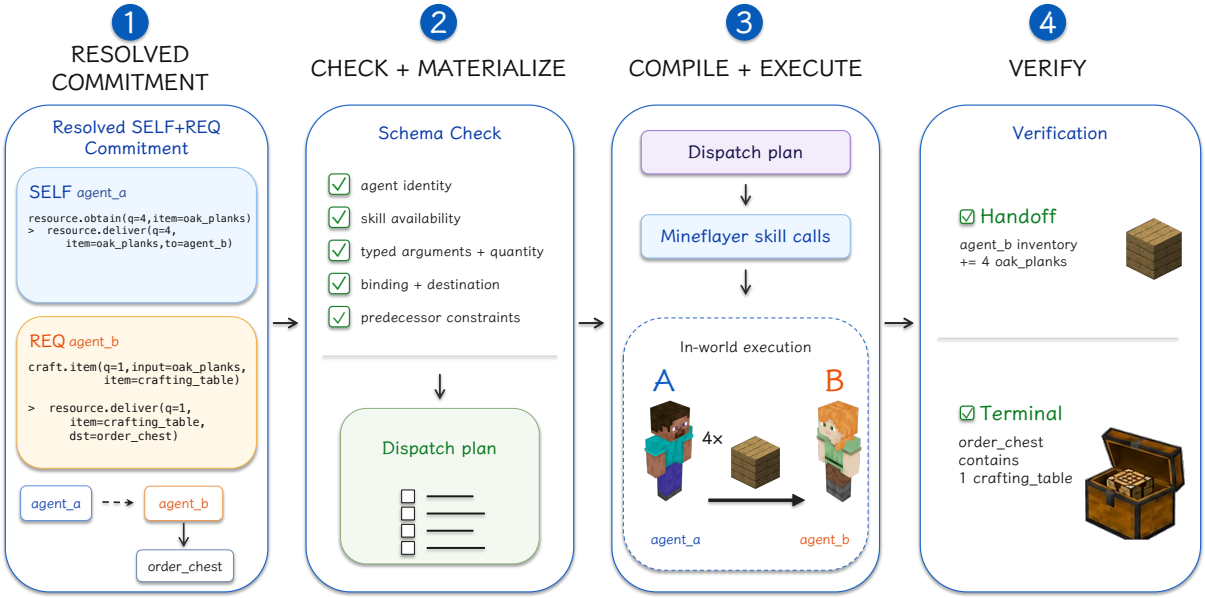


Figure 2: Making an obligation observable. A resolved object is checked, materialized as a dispatch plan, and executed through Minecraft skills. Inventory readback verifies that the receiver obtained the planks and that the crafting table reached the order chest. These checks establish fulfillment of the commitment.

views. For example, Recipe changes what the receiver makes from planks, whereas Allocation changes whether the receiver stores material or uses it for construction. We hold the skill library and task checks fixed across communication methods so their cost can be compared on the same physical work. The workcell diagnostic crosses two goals with two prescribed processing modes, both of whose crafting routes are physically executable. In the controlled local revision diagnostic, a parsed initial proposal must match the prescribed proposal before the second call begins.

We examine task structure by keeping the instance graph private to the sender and transmitting a continuation produced by deterministic projection. These physical tests cover 114 parameter settings within 24 template–motif families, and a distinct probe measures the unchanged Direct model's ability to generate the object. Language tests use 32 original phrasings across 16 assignments or 640 new test descriptions, with 40 per assignment and 256 disjoint calibration descriptions. We evaluate the existing checkpoints on both description sets. The selector chooses between each template's two assignments, after which the same protocol handles the selected work.

5.2 Models and Protocol Comparisons

Our local backend is Qwen3.5-0.8B, trained with programmatic examples of local facts and task paths [42]. Direct uses 1,920 training examples, two epochs, and 240 updates per seed, with effective batch size 16 and deterministic decoding. We train three checkpoints independently and compare complete training and control procedures for the two horizons, giving single-step control 2,880 examples and 360 updates to supervise its additional intermediate decision. JSON encodes the same fields and uses the same two call stages as Short DSL. Controlled free-form includes dialogue, accumulated context, and structured extraction, with 2,400 training examples and 300 updates. Factor predicts a short assignment code using matched training information.

For the prompted control, we use the same DeepSeek-V4.1-Flash service for both protocols [15], with four worked examples balanced across the two role assignments. Dialogue examples use the teacher conversations from the local free-form training, identify both agents' public roles, and begin with the supplier. Within eight calls, the agents exchange natural language and submit structured proposals and acceptances whose explicit control markers the controller recognizes independently of explanatory prose. We test 128 task clusters with two roles and three repetitions per protocol. Feedback tests use 40 paired workcell units, comparing revision from a supplied proposal with the full interaction. Embedding retrieval uses Qwen3-Embedding-0.6B [61], and the supervised lexical classifier is fitted on the training set. These selectors represent practical choices with different pretraining and supervision (Supplement §S3–S6).

5.3 Measurements and Comparisons

We measure commitment formation, verified handoff H, fulfillment F, task success G, and the stage where a trial stops, including all assigned trials in task-success rates. Time to commitment (TTC) includes proposal generation and processing up to commitment formation or a stopped attempt, while episode time includes world execution. We measure peer traffic, model calls, input and output tokens, and token-based API charges. For local structured messages, peer payload is the request, whereas free-form payload includes

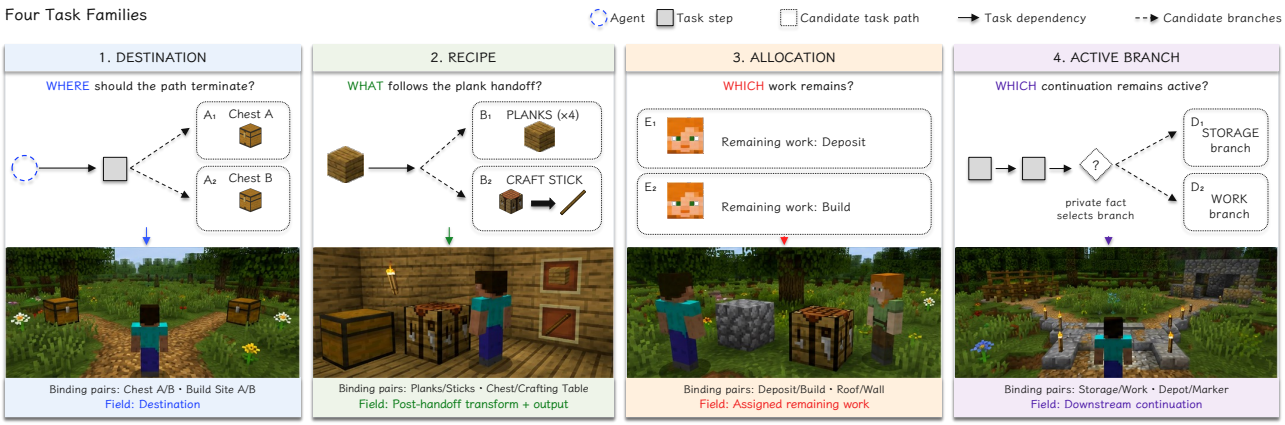


Figure 3: Four families vary the executable consequence of a private fact: destination, post-handoff transformation and output, assigned remaining work, or downstream continuation. Both Recipe templates transfer oak planks. The Allocation alternatives change storage/build work or the assigned wall/roof component.

peer dialogue. The private-graph comparison counts application requests and acknowledgments under a common accounting rule.

Local surface and backend comparisons each use a 128-cluster quality pool and a distinct 40-cluster timing sample. Methods run sequentially on one RTX 4090D with hot models and batch size one. The DeepSeek comparison has 768 quality trials per protocol and 64 fresh timing trials per protocol, with the two protocols interleaved in a fixed alternating order, HTTP concurrency one, and one world at a time. Its TTC includes API service and network time. The receiver benchmark measures the receiver stage on a common local device. Selector accuracy is evaluated before world execution on 640 descriptions, while the decision-time comparison uses 128 measurements per strategy.

We report 95% intervals from 10,000 paired bootstrap draws over task clusters, averaging roles and repeated calls within each unit [17]. For language comparisons, we resample source phrasings within their 16 assignments. Message substitutions preserve the world, local inputs, and call procedure while changing the legal request or response. Their role is to check the protocol, alongside fault injections at admission, handoff, and completion.

6 Results

We test how path encoding, commitment length, and receiver implementation affect the model computation needed for coordination. Physical checks distinguish fulfillment from task success.

6.1 Communication and Model Work

Compact paths reduce the work of expressing a commitment while preserving its task fields. All three local representations complete the 128-cluster quality pool (Table 1). The full-key JSON control uses the same fields and call stages as Short DSL, yet its output length and delay approach those of controlled free-form communication (Figure 4(a)). Short DSL expresses these fields as skill paths with typed arguments and an explicit precedence separator, reducing peer traffic by 92.8%, median TTC by 68.2%, and median episode time by 17.4% relative to free-form communication. The encoding thus changes the model work needed to express the same task information. GenCoord defines the joint work, handoff dependencies, and commitment checks, while Short DSL makes this object compact to generate.

With the same DeepSeek backend, Short DSL completes 768/768 quality trials and dialogue completes 766/768 (Table 2). Dialogue uses 8.1 times the peer traffic and 2.8 times the output tokens (Figure 4(b)). The protocols make a similar number of calls, but generate messages of substantially different lengths. Short DSL specifies the requested work in the resolver's input format.

In the interleaved timing sample, Short DSL reduces mean TTC by 30.4% and median TTC by 20.8%. All timed trials form commitments, and task success is 63/64 for Short DSL and 64/64 for dialogue because one Short DSL handoff fails after formation. The hosted timing reduction is smaller than the local reduction, although transmitted messages and generated outputs are substantially shorter in both deployments. Hosted TTC includes service and network time as well as token processing, so we evaluate each protocol's delay in its actual deployment (Supplement §S5–S6).

6.2 Multistep Work and Feedback

Compact encoding reduces the model work spent expressing a plan, while a longer commitment changes how often the agents must decide again. We test this second choice by extending the committed path beyond the next handoff or intermediate decision. On held-out templates, multistep control reaches 98.1% task success against 91.3% for single-step control, a paired improvement of 6.9 points [2.9, 10.8]. Mean online decisions fall from 2.91 to 1.98, a 32% reduction (Figure 4(c,d)). Multistep control commits past the intermediate stage and uses fewer training examples and updates. The result extends multi-step intent from guiding partner search [5] to carrying out a committed sequence, showing that longer commitments improve generalization with fewer decisions.

Feedback lets the requester revise that sequence when its partner has private information about the assignment. Given an initial proposal and correct feedback, DeepSeek completes 419/480 workcell

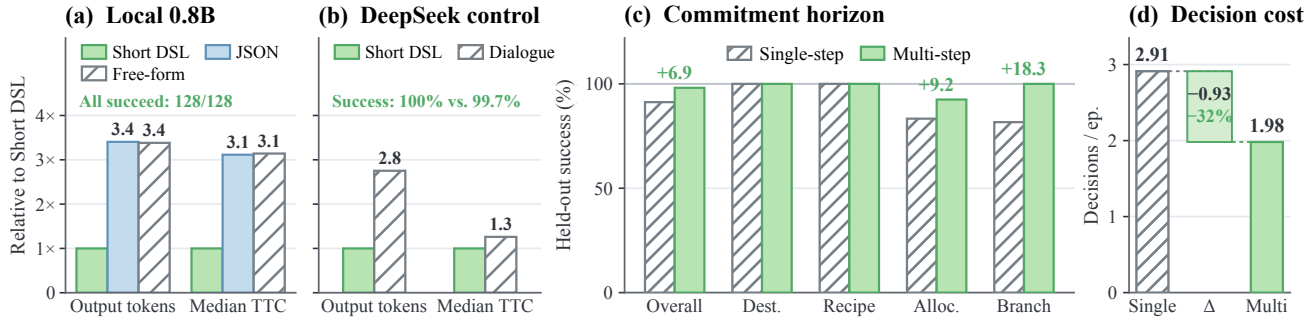


Figure 4: Communication costs relative to Short DSL and commitment horizon. (a) Local JSON and controlled free-form have similar output-token and median-TTC ratios, and all three complete the quality pool. (b) DeepSeek dialogue's output-token ratio is larger than its median-TTC ratio. Bars in (a,b) share a scale, with Short DSL at 1. Panel (a) uses the 40-cluster timing sample; panel (b) combines quality-pool output means with medians from 64 timing trials per protocol. (c,d) Multistep commitments improve held-out-template success and reduce online decisions.

Table 1: Communication surfaces at matched task success. Quality and episode time use the 128-cluster pool; online costs use a 40-cluster surface-comparison timing sample. Wire is measured peer payload; TTC ends at commitment formation. p50/p95 denote median/95th percentile.

Surface	Success	Calls	Input tok.	Output tok.	Wire (B)	TTC p50/p95 (s)	Episode p50/p95 (s)
Short DSL	128/128	2.00	949.1	78.6	76.3	2.27 / 2.62	24.86 / 27.06
JSON	128/128	2.00	1154.7	267.6	303.4	7.07 / 7.86	29.94 / 31.82
Controlled free-form	128/128	2.50	1585.2	265.8	1061.1	7.13 / 7.86	30.09 / 31.92

Table 2: Communication protocols with the same DeepSeek model. Quality uses 768 trials per protocol; TTC uses 64 interleaved trials.

Metric	Short DSL	Dialogue
Task success	768/768	766/768
Calls	2.00	2.12
Peer payload (B)	112.75	913.76
Output tokens	100.4	276.3
Mean TTC (s)	4.29	6.17
Median TTC (s)	4.38	5.52

trials (87.3%); when it generates the initial proposal as well, it completes 414/480 (86.3%). The revised commitment changes who crafts and which item crosses the handoff, while preserving the requested product. Correct versus alternative feedback has a task-success difference of 87.3 points [84.4, 90.0]. Alternative feedback changes the workcell's processing assignment: DeepSeek fulfills all 403 formed commitments and delivers the product to its target, while original-task success is zero across 480 trials. A compact response thus revises both agents' work, while fulfillment and original-task checks reveal whether the resulting assignment meets the task.

6.3 Selection and Receiver Implementation

A complete requested path also makes reception an implementation choice: the receiver can instantiate the work directly instead of generating a continuation. On paired sender outputs, both receivers form the same commitments. Task-success counts differ by two Factor trials and three Direct trials, with all differences arising after formation. Mean calls fall from approximately 1.94–1.95 to one, saving 343 ms [292, 374] with Factor and 785 ms with Direct before commitment. Because both implementations subsequently dispatch the work expressed by the request, the comparison separates the model needed to select an assignment from the code needed to realize its specified continuation.

To test the receiver's required knowledge, we keep the sender's instance graph private and supply deterministically projected paths, which complete 456 physical cases. Goal-based search supplies another 100 different legal continuations that also fulfill the commitment and task. GOAL sends fewer first-delivery bytes than PATH, 662 versus 741 including acknowledgment, but uses 2.0 rather than 0.065 ms of receiver computation. The unchanged Direct model forms 43.1% of complete objects in the corresponding generation probe. The projected-path result establishes the receiver's ability to execute supplied structure, while that probe measures the sender's ability to express it.

The fulfillment and task checks used in the feedback intervention also locate errors in commitments formed from descriptions. In closed-loop runs based on 32 original descriptions, 2,181 of the 2,198 task failures after formation follow fulfillment of an incorrect assignment, while 17 arise during execution.

We therefore examine the deployment tradeoff where a model interprets a task description and selects an assignment (Table 3). On 640 new descriptions over the shared task vocabulary, the local

Table 3: Selecting an assignment from new descriptions. Accuracy is measured before execution on 640 descriptions (3,840 choices for DeepSeek). Mean decision time comes from 128 timed decisions per strategy and includes service and network time for API calls.

Selector	Accuracy (%)	Time (ms)
Embedding retrieval	82.19	92
Factor (local 0.8B)	95.94	412
DeepSeek	99.97	1,402
Retrieval + DeepSeek	88.91	350

Factor model selects correctly in 95.94% of trials at a mean decision time of 412 ms, compared with 99.97% at 1,402 ms for DeepSeek. Factor uses about 29% of the hosted model's measured decision time and makes no API request, whereas DeepSeek supplies the highest assignment accuracy. Routing the lowest-margin retrieval decisions to DeepSeek raises accuracy from 82.19% to 88.91%; the same-quota random-upgrade expectation is 84.88%. The routing point uses the hosted model selectively, while running Factor throughout gives higher accuracy with local computation.

Each selector expresses its chosen assignment through the same receiver-facing interface, so we can change how work is chosen while keeping its realization and checking procedures fixed. The local and hosted results supply concrete operating points for accuracy, response time, and dependence on an external service.

7 Discussion

Communication spends inference. Making joint work explicit reduces inference at several points before the next physical action. Compact paths reduce what agents generate, complete continuations support deterministic reception, and multistep commitments reduce how often the agents decide again. The full-key JSON control varies how task fields are encoded. The other comparisons vary how the receiver processes a received path and how many steps each commitment covers. Together, these comparisons show why a protocol should specify executable work, not only its transmission.

Figure 4(a,b) shows why generated output and elapsed time provide different views of communication cost. Local output length and commitment time change by similar proportions. With the hosted model, however, the timing difference is smaller, which may partly reflect service and network overhead as well as prompt processing. We therefore measure both the work required to produce a commitment and the delay before the agents can use it. Across both deployments, compact paths keep the actors, transfer, and continuation explicit while reducing generation.

Models choose, skills realize. Interpreting a description and instantiating a complete continuation are different operations. A language model can select or revise an assignment, while shared skills and a deterministic receiver realize work that the message has already specified. Changing the selector changes assignment accuracy and decision time, while the interface for sending, executing, and checking the selected work stays fixed. The local selector provides a fast, self-contained operating point, and the hosted selector provides higher assignment accuracy. We check the requested work against the commitment and the chosen assignment against the task, distinguishing fulfillment from task success.

Commitments connect both agents' work. The defining object in GenCoord couples the sender's promised delivery to the receiver's work after receipt. This gives decentralized proposals the dependency structure that a centrally constructed joint plan would otherwise provide, while each proposal is generated from the participant's local information. Conditional commitment semantics explains when responsibilities take effect, and physical readback establishes whether the promised actions occurred. Admission, fulfillment, and task checks then serve different engineering purposes: validate the proposed work, detect an incomplete transfer or outcome, and assess whether the selected work satisfies the task. The same trace can consequently guide improvements to the assignment model, the protocol checks, or the physical skill that did not complete its work.

Knowledge, messages, and search. A path transmits a selected solution, while a goal and constraints ask the receiver to find one. A shared directory moves recurring task knowledge into the receiver's prior, and Factor's composer expands a short internal prediction into an explicit partner request. These alternatives show why the shortest message and the least computation need not coincide. Designers can choose how much structure to transmit by considering the receiver's knowledge and search cost together with the model work needed to express the message. Each commitment specifies who performs each step, what is handed over, and which outcome the receiver must achieve after receipt.

Limitations. We evaluate bilateral assignments in static Minecraft worlds with reliable messaging, shared discrete skills, and paired task alternatives. Extending the protocol to changing environments, larger teams, open-ended task decomposition, and robot platforms with continuous control and perception errors will require corresponding models of revision and responsibility. The static tests leave the cost of revising long commitments after environmental changes unmeasured.

8 Conclusion

We introduced GenCoord to let embodied agents communicate their own work and a partner's continuation as commitments. Its compact skill paths reduce communication and model inference costs with a local 0.8B backend, and communicating those paths through the prompted cloud model also reduces generated output and commitment time. Multistep commitments improve task success on held-out templates while reducing decisions, and structured feedback supports changes to the proposed assignment. Because the receiver can instantiate a complete request deterministically, we can choose between local and hosted selectors according to deployment needs while keeping the partner's execution interface unchanged. After the skills execute the committed paths, we check whether the agents fulfilled the commitment and whether their work completed the original task. Making the division of work the message lets models choose and revise that work and lets skills carry it out.

References

[1] Cristian-Paul Bara, Sky CH-Wang, and Joyce Chai. 2021. MindCraft: Theory of Mind Modeling for Situated Dialogue in Collaborative Tasks. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1112–1125. doi:10.18653/v1/2021.emnlp-main.85

[2] Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. 2002. The Complexity of Decentralized Control of Markov Decision Processes. *Mathematics of Operations Research* 27, 4 (2002), 819–840. doi:10.1287/moor.27.4.819.297

[3] Neel P. Bhatt, Po-han Li, Kushagra Gupta, Rohan Siva, Daniel Milan, Alexander T. Hogue, Sandeep P. Chinchali, David Fridovich-Keil, Zhangyang Wang, and Ufuk Topcu. 2026. UNCAP: Uncertainty-Guided Neurosymbolic Planning Using Natural Language Communication for Cooperative Autonomous Vehicles. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2609–2617. doi:10.65109/WHRE3513

[4] Qi Chai, Zhang Zheng, Junlong Ren, Deheng Ye, Zichuan Lin, and Hao Wang. 2025. CausalMACE: Causality Empowered Multi-Agents in Minecraft Cooperative Tasks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 14410–14426. doi:10.18653/v1/2025.findings-emnlp.777

[5] Shenghui Chen, Ruihan Zhao, Sandeep Chinchali, and Ufuk Topcu. 2025. Human-Agent Coordination in Games under Incomplete Information via Multi-Step Intent. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, Detroit, Michigan, USA, 481–489. https://www.ifaamas.org/Proceedings/aamas2025/pdfs/p481.pdf

[6] Weize Chen, Chenfei Yuan, Jiarui Yuan, Yusheng Su, Chen Qian, Cheng Yang, Ruobing Xie, Zhiyuan Liu, and Maosong Sun. 2024. Beyond Natural Language: LLMs Leveraging Alternative Formats for Enhanced Reasoning and Communication. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 10626–10641. doi:10.18653/v1/2024.findings-emnlp.623

[7] Weize Chen, Jiarui Yuan, Chen Qian, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2025. Optima: Optimizing Effectiveness and Efficiency for LLM-Based Multi-Agent System. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 11534–11557. doi:10.18653/v1/2025.findings-acl.601

[8] Yongchao Chen, Jacob Arkin, Yang Zhang, Nicholas Roy, and Chuchu Fan. 2024. Scalable Multi-Robot Collaboration with Large Language Models: Centralized or Decentralized Systems?. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 4311–4317. doi:10.1109/ICRA57147.2024.10610676

[9] Han-Lim Choi, Luc Brunet, and Jonathan P. How. 2009. Consensus-Based Decentralized Auctions for Robust Task Allocation. *IEEE Transactions on Robotics* 25, 4 (2009), 912–926. doi:10.1109/TRO.2009.2022423

[10] Amit K. Chopra, Matteo Baldoni, Samuel H. Christie V, and Munindar P. Singh. 2025. Azorus: Commitments over Protocols for BDI Agents. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 490–499. https://www.ifaamas.org/Proceedings/aamas2025/pdfs/p490.pdf

[11] Amit K. Chopra, Samuel H. Christie, V, and Munindar P. Singh. 2026. Peach: Program Each Agent and Communicate Howsoever. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 1022–1030. doi:10.65109/SAXY3841

[12] Samuel H. Christie, V, Amit K. Chopra, and Munindar P. Singh. 2022. Pippi: Practical Protocol Instantiation. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, Online, 281–289. https://www.ifaamas.org/Proceedings/aamas2022/pdfs/p281.pdf

[13] Samuel H. Christie, V, Munindar P. Singh, and Amit K. Chopra. 2023. Kiko: Programming Agents to Enact Interaction Protocols. In *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 1154–1163. https://www.ifaamas.org/Proceedings/aamas2023/pdfs/p1154.pdf

[14] Philip R. Cohen and Hector J. Levesque. 1991. Teamwork. *Noûs* 25, 4 (1991), 487–512. doi:10.2307/2216075

[15] DeepSeek. 2026. DeepSeek API Documentation. Documentation accessed September 29, 2026. https://api-docs.deepseek.com/

[16] Yubo Dong, Xukun Zhu, Zhengzhe Pan, Linchao Zhu, and Yi Yang. 2024. VillagerAgent: A Graph-Based Multi-Agent Framework for Coordinating Complex Task Dependencies in Minecraft. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 16290–16314. doi:10.18653/v1/2024.findings-acl.964

[17] Bradley Efron and Robert J. Tibshirani. 1994. *An Introduction to the Bootstrap* (1 ed.). Chapman and Hall/CRC, New York. doi:10.1201/9780429246593

[18] Amy Fang and Hadas Kress-Gazit. 2024. High-Level, Collaborative Task Planning Grammar and Execution for Heterogeneous Agents. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, Auckland, New Zealand, 544–552. https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p544.pdf

[19] Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. 2016. Learning to Communicate with Deep Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 29.

[20] Nicoletta Fornara and Marco Colombetti. 2002. Operational Specification of a Commitment-Based Agent Communication Language. In *Proceedings of the First International Joint Conference on Autonomous Agents and Multiagent Systems*. ACM. doi:10.1145/544862.544868

[21] Brian P. Gerkey and Maja J. Matarić. 2004. A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems. *The International Journal of Robotics Research* 23, 9 (2004), 939–954. doi:10.1177/0278364904045564

[22] Ran Gong, Qiuyuan Huang, Xiaojian Ma, Yusuke Noda, Zane Durante, Zilong Zheng, Demetri Terzopoulos, Li Fei-Fei, Jianfeng Gao, and Hoi Vo. 2024. MindAgent: Emergent Gaming Interaction. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, 3154–3183. doi:10.18653/v1/2024.findings-naacl.200

[23] Barbara J. Grosz and Sarit Kraus. 1996. Collaborative Plans for Complex Group Action. *Artificial Intelligence* 86, 2 (1996), 269–357. doi:10.1016/0004-3702(95)00103-4

[24] Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L. Griffiths, and Mengdi Wang. 2026. Embodied LLM Agents Learn to Cooperate in Organized Teams. *IEEE Transactions on Computational Social Systems* 13, 2 (2026), 2514–2530. doi:10.1109/TCSS.2025.3637527

[25] Shashwat Gupta, Anson Bastos, Mayukh Das, Supriyo Ghosh, Nagarajan Natarajan, Chetan Bansal, and Saravan Rajmohan. 2026. Learning Optimal Message Representations for Agentic Communication. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 28849–28879. doi:10.18653/v1/2026.findings-acl.1441

[26] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html

[27] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, et al. 2023. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Proceedings of The 6th Conference on Robot Learning (Proceedings of Machine Learning Research, Vol. 205)*. PMLR, 287–318. https://proceedings.mlr.press/v205/ichter23a.html

[28] Natasha Jaques, Angeliki Lazaridou, Edward Hughes, Caglar Gulcehre, Pedro Ortega, Dj Strouse, Joel Z. Leibo, and Nando De Freitas. 2019. Social Influence as Intrinsic Motivation for Multi-Agent Deep Reinforcement Learning. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 3040–3049. https://proceedings.mlr.press/v97/jaques19a.html

[29] Nicholas R. Jennings, Peyman Faratin, Alessio R. Lomuscio, Simon Parsons, Michael J. Wooldridge, and Carles Sierra. 2001. Automated Negotiation: Prospects, Methods and Challenges. *Group Decision and Negotiation* 10, 2 (2001), 199–215. doi:10.1023/A:1008746126376

[30] HuaDong Jian, Chenghao Li, Haoyu Wang, Jiajia Shuai, Jinyu Guo, Yang Yang, and Chaoning Zhang. 2026. Gated Coordination for Efficient Multi-Agent Collaboration in Minecraft Game. *arXiv preprint arXiv:2604.18975* (2026). https://arxiv.org/abs/2604.18975

[31] Shyam Sundar Kannan, Vishnunandan L. N. Venkatesh, and Byung-Cheol Min. 2024. SMART-LLM: Smart Multi-Agent Robot Task Planning using Large Language Models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. doi:10.1109/IROS58592.2024.10802322

[32] Hojoon Kim, Yuheng Wu, and Thierry Tambe. 2026. AgenticCache: Cache-Driven Asynchronous Planning for Embodied AI Agents. In *Proceedings of Machine Learning and Systems*, Vol. 8.

[33] Huao Li, Yu Quan Chong, Simon Stepputtis, Joseph Campbell, Dana Hughes, Michael Lewis, and Katia Sycara. 2023. Theory of Mind for Multi-Agent Collaboration via Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 180–192. doi:10.18653/v1/2023.emnlp-main.13

[34] Yaoru Li, Shunyu Liu, Tongya Zheng, Li Sun, and Mingli Song. 2026. Parallelized Planning-Acting for Multi-Agent LLM Systems in Minecraft. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 155–164. doi:10.65109/EXAJ9853

[35] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as Policies: Language Model Programs for Embodied Control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 9493–9500. doi:10.1109/ICRA48891.2023.10160591

[36] Kehui Liu, Zixin Tang, Dong Wang, Zhigang Wang, Xuelong Li, and Bin Zhao. 2025. COHERENT: Collaboration of Heterogeneous Multi-Robot System with Large Language Models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. doi:10.1109/ICRA55743.2025.11127808

[37] Qian Long, Zhi Li, Ran Gong, Ying Nian Wu, Demetri Terzopoulos, and Xiaofeng Gao. 2024. TeamCraft: A Benchmark for Multi-Modal Multi-Agent Systems in Minecraft. *arXiv preprint arXiv:2412.05255* (2024). https://arxiv.org/abs/2412.05255

[38] Ryan Lowe, Jakob Foerster, Y-Lan Boureau, Joelle Pineau, and Yann Dauphin. 2019. On the Pitfalls of Measuring Emergent Communication. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 693–701. doi:10.5555/3306127.3331757

[39] Arnab Mallick and Indraveni Chebolu. 2026. μACP: A Formal Calculus for Expressive, Resource-Constrained Agent Communication. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 50–58. doi:10.65109/PHRW6922

[40] Zhao Mandi, Shreeya Jain, and Shuran Song. 2024. RoCo: Dialectic Multi-Robot Collaboration with Large Language Models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 286–299. doi:10.1109/ICRA57147.2024.10610855

[41] PrismarineJS. 2026. Mineflayer: Create Minecraft Bots with a High-Level JavaScript API. Software repository. Version 4.37.1 used in the project; accessed 2026-08-08. https://github.com/PrismarineJS/mineflayer

[42] Qwen Team. 2026. Qwen3.5-0.8B. Hugging Face model card. Accessed 2026-08-08. https://huggingface.co/Qwen/Qwen3.5-0.8B

[43] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. 2025. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv preprint arXiv:2506.01844* (2025). doi:10.48550/arXiv.2506.01844

[44] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. ProgPrompt: Program Generation for Situated Robot Task Planning Using Large Language Models. *Autonomous Robots* 47 (2023), 999–1012. doi:10.1007/s10514-023-10135-3

[45] Munindar P. Singh. 1998. Agent Communication Languages: Rethinking the Principles. *Computer* 31, 12 (1998), 40–47. doi:10.1109/2.735849

[46] Munindar P. Singh. 2008. Semantical Considerations on Dialectical and Practical Commitments. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*. AAAI Press, 176–181. https://cdn.aaai.org/AAAI/2008/AAAI08-028.pdf

[47] Munindar P. Singh. 2011. Information-Driven Interaction-Oriented Programming: BSPL, the Blindingly Simple Protocol Language. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 491–498. https://www.ifaamas.org/Proceedings/aamas2011/papers/A4_B57.pdf

[48] Munindar P. Singh and Amit K. Chopra. 2020. Clouseau: Generating Communication Protocols from Commitments. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. AAAI Press, 7244–7252. doi:10.1609/aaai.v34i05.6215

[49] Reid G. Smith. 1980. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Trans. Comput.* C-29, 12 (1980), 1104–1113. doi:10.1109/TC.1980.1675516

[50] Peter Stone, Gal A. Kaminka, Sarit Kraus, and Jeffrey S. Rosenschein. 2010. Ad Hoc Autonomous Agent Teams: Collaboration without Pre-Coordination. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*. AAAI Press, 1504–1509. doi:10.1609/aaai.v24i1.7529

[51] Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. 2016. Learning Multiagent Communication with Backpropagation. In *Advances in Neural Information Processing Systems*, Vol. 29.

[52] Milind Tambe. 1997. Towards Flexible Teamwork. *Journal of Artificial Intelligence Research* 7 (1997), 83–124. doi:10.1613/jair.433

[53] Baiqing Wang, Helei Cui, Bo Zhang, Xiaolong Zheng, Bin Guo, and Zhiwen Yu. 2026. MeCo: Enhancing LLM-Empowered Multi-Robot Collaboration via Similar Task Memoization. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 32–40. doi:10.65109/QPUC1624

[54] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=ehfRiF0R3a

[55] Rundong Wang, Xu He, Runsheng Yu, Wei Qiu, Bo An, and Zinovi Rabinovich. 2020. Learning Efficient Multi-agent Communication: An Information Bottleneck Approach. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 9908–9918.

[56] Isadora White, Kolby Nottingham, Ayush Maniar, Max Robinson, Hansen Lillemark, Mehul Maheshwari, Lianhui Qin, and Prithviraj Ammanabrolu. 2025. Collaborating Action by Action: A Multi-agent LLM Framework for Embodied Reasoning. *arXiv preprint arXiv:2504.17950* (2025). https://arxiv.org/abs/2504.17950

[57] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*. https://openreview.net/forum?id=BAakY1hNKS

[58] Juheon Yi, Jinglu Wang, Xiaoyi Zhang, and Yan Lu. 2026. Multi-agent Framework for Time-Sensitive Complementary Collaboration in Minecraft. *arXiv preprint arXiv:2606.15684* (2026). https://arxiv.org/abs/2606.15684

[59] Pınar Yolum and Munindar P. Singh. 2002. Flexible Protocol Specification and Execution: Applying Event Calculus Planning Using Commitments. In *Proceedings of the First International Joint Conference on Autonomous Agents and Multiagent Systems*. ACM, 527–534. doi:10.1145/544862.544867

[60] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B. Tenenbaum, Tianmin Shu, and Chuang Gan. 2024. Building Cooperative Embodied Agents Modularly with Large Language Models. In *The Twelfth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/54b8b4e0b4ba4aad112e84f32e3b5dbb-Abstract-Conference.html

[61] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025). doi:10.48550/arXiv.2506.05176

[62] Yang Zhang, Shixin Yang, Chenjia Bai, Fei Wu, Xiu Li, Zhen Wang, and Xuelong Li. 2025. Towards Efficient LLM Grounding for Embodied Multi-Agent Collaboration. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 1663–1699. doi:10.18653/v1/2025.findings-acl.84

[63] Tan Zhi-Xuan, Lance Ying, Vikash Mansinghka, and Joshua B. Tenenbaum. 2024. Pragmatic Instruction Following and Goal Assistance via Cooperative Language-Guided Inverse Planning. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2094–2103. https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p2094.pdf

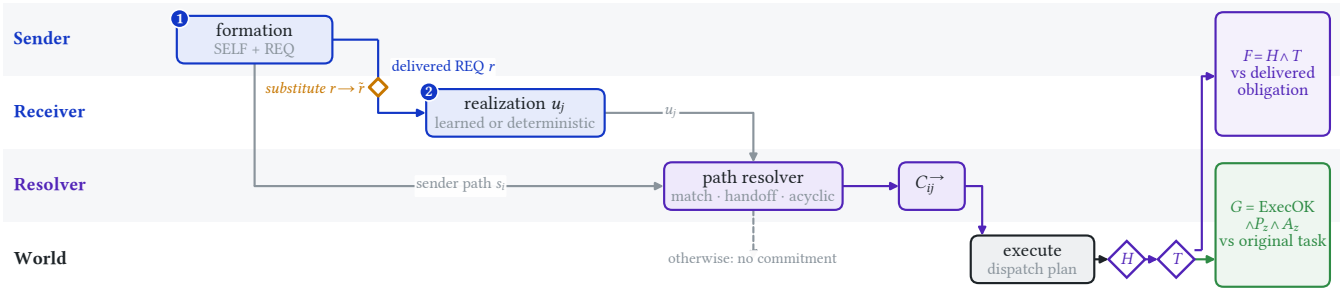
Appendix

Sections S1–S4 describe the protocol, tasks, and evaluation units. S5–S6 report local and prompted coordination, S7–S11 examine implementation and assignment choices, and S12–S13 provide protocol diagnostics and structural tests.

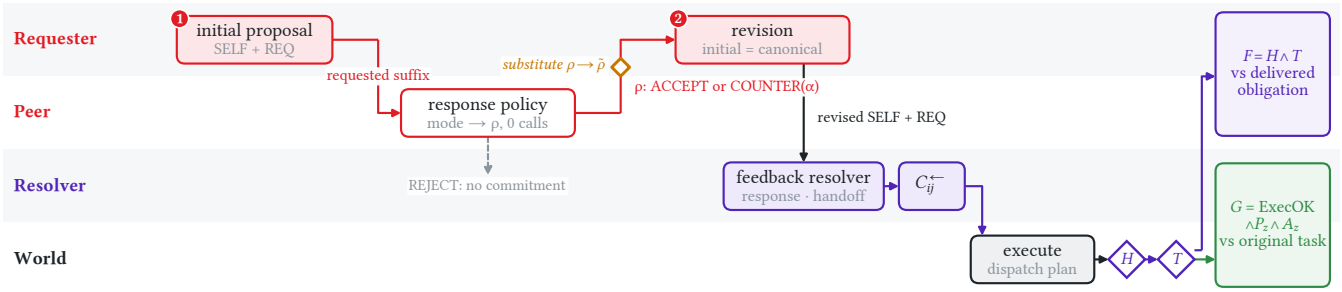
S1 Protocol Lifecycles and Model Calls

This section supports the commitment protocol in main Section 4 by specifying how proposed paths become commitments and how the runtime checks their execution. We provide the complete call sequence and resolver conditions so that the learned and deterministic implementations can be compared against the same semantics.

Figure S1 shows where generation occurs in each interaction and where a delivered message can be replaced before resolution. Its final panel compares implementations that share the same checks while using different numbers of model calls.



(a) Forward request



(b) Feedback

	Learned calls	Resolved commitment	World checks H, T	Original-task score G
Learned forward (Direct or Factor)	● ●	●	●	●
Forward with deterministic receiver	● ○	●	●	●
Zero-call binding rule	○ ○	●	●	●
Feedback, ACCEPT or COUNTER	● ●	●	●	●
Feedback, REJECT	● ○	—	—	● as failure

(c) Learned calls and checks by configuration

Figure S1. How a proposal becomes executable joint work. In the forward route (a), the receiver realizes a delivered request through generation or deterministic instantiation before the resolver checks compatibility and dependencies. In the feedback route (b), the deterministic response selects acceptance, a counteroffer, or rejection, and the requester revises an accepted or countered proposal. Physical checks evaluate receipt H and declared completion T, distinguishing fulfillment F from original-task success G. Panel (c) locates model calls and checks across implementations.

We use the notation in Table S1 throughout the algorithms, including the distinction between proposed paths and the resolved commitment. Single-step control adds a sender decision after acquisition, whereas the two-call Factor implementation predicts a code at each side and composes the corresponding paths deterministically.

To keep the evaluator's outcome distinct from the state of an individual obligation, we return the following tagged result and report fulfillment alongside original-task success:

$$\mathcal{R} = (\{\texttt{SUCCESS}\} \times \mathcal{C} \times \mathcal{D}) \uplus (\{\texttt{NO_COMMITMENT}\} \times \mathcal{E}) \uplus (\{\texttt{FAIL}\} \times \mathcal{F}).$$

Table S1. Notation for proposals, resolution, dispatch, and world evaluation in Algorithms S1–S2.

Symbol	Meaning and domain
$\mathcal{A} = \{a_i, a_j\}$	the two agents; protocol roles are written as *snd*, *rcv*, *req*, and *peer*
$\mathcal{K} = (\mathcal{S}, \mathcal{T}, \Theta, \Gamma)$	core executable prior: skills $\mathcal{S}$, shared task DAG $\mathcal{T}$, schema Θ, and grounding/checker contract Γ; in the private-structure setting only primitive semantics $\mathcal{K}_0$ are shared
ξ_{pub}, ω_t, x_i^t	public episode instantiation, current executable world state, and agent i's model-visible local context at round t; ω_t is visible only to the runtime verifier
$\hat{y}_{\text{snd}}, q_{\text{snd}}$; $\hat{y}_{\text{rcv}}, q_{\text{rcv}}$	raw model strings and parsed sender/receiver proposals; $q = \perp$ denotes parse failure
$r_{\text{snd}\to\text{rcv}}$, u_{rcv}	requested peer path and receiver-local realization suffix
c_{peer}, ρ, α	peer-local processing mode (data field `capability`), bounded response, and selected `counter_offer_id`
$N_{\mathcal{K}}^{\text{path}}$, $N_{\mathcal{K}}^{\text{prop}}$, Actorize	canonical normalization for paths/proposals and explicit instantiation of an implicit task actor
$\mathcal{C}, \mathcal{D}, \mathcal{E}, \mathcal{F}$	spaces of resolved commitments, dispatch plans, no-commitment reasons, and protocol/runtime failure codes

The values $\text{Success}(C, D)$, $\text{NoCommitment}(e)$, and $\text{Fail}(f)$ identify whether a task completes or stops at an earlier stage. Because fulfilling the committed work can still violate the original assignment, a task-failure result may include $H = F = 1$ and $G = 0$.

Algorithm S1 Learned Forward Resolution

Require: sender context x_{snd}, receiver context x_{rcv}, prior $\mathcal{K}$, public episode instantiation ξ_{pub}, executable world ω_t

$\hat{y}_{\text{snd}} \leftarrow \textsc{Generate}(x_{\text{snd}}, \mathcal{K}, \xi_{\text{pub}})$ ▷ SELF+REQ
$q_{\text{snd}} \leftarrow \textsc{Parse}(\hat{y}_{\text{snd}})$
if $q_{\text{snd}} = \perp$ **then**
  return $\textsc{Fail}(\texttt{PARSE_FAILURE})$
if $\neg\textsc{ContractCheck}(q_{\text{snd}}, x_{\text{snd}}, \mathcal{K}, \xi_{\text{pub}})$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
if $|q_{\text{snd}}.\texttt{peer_requests}| \neq 1$ **or** $q_{\text{snd}}.\texttt{peer_requests}[0].\texttt{requested_plan} = [\,]$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
$r_{\text{snd}\to\text{rcv}} \leftarrow q_{\text{snd}}.\texttt{peer_requests}[0]$; deliver $r_{\text{snd}\to\text{rcv}}$
$\hat{y}_{\text{rcv}} \leftarrow \textsc{Generate}(x_{\text{rcv}}, \mathcal{K}, \xi_{\text{pub}}, r_{\text{snd}\to\text{rcv}})$
$q_{\text{rcv}} \leftarrow \textsc{Parse}(\hat{y}_{\text{rcv}})$
if $q_{\text{rcv}} = \perp$ **then**
  return $\textsc{Fail}(\texttt{PARSE_FAILURE})$
if $\neg\textsc{ContractCheck}(q_{\text{rcv}}, x_{\text{rcv}}, \mathcal{K}, \xi_{\text{pub}})$ **or** $q_{\text{rcv}}.\texttt{self_plan} = [\,]$ **or** $q_{\text{rcv}}.\texttt{peer_requests} \neq [\,]$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
$C \leftarrow \textsc{Resolve}_{\to}(q_{\text{snd}}, q_{\text{rcv}}, \mathcal{K}, \xi_{\text{pub}})$
if $C = \perp$ **then**
  return $\textsc{Fail}(\texttt{RESOLUTION_CONFLICT})$
$D \leftarrow \textsc{Materialize}(C, \mathcal{K}, \xi_{\text{pub}})$
if $D = \perp$ **then**
  return $\textsc{Fail}(\texttt{MATERIALIZATION_FAILURE})$
$\texttt{status} \leftarrow \textsc{ExecuteAndVerify}(D, \omega_t)$
if $\texttt{status} \neq \texttt{SUCCESS}$ **then**
  return $\textsc{Fail}(\texttt{status})$
return $\textsc{Success}(C, D)$

The response policy deterministically maps the requested suffix and processing mode to Accept, Reject, or $\textsc{Counter}(\alpha)$, whose alternative is serialized as `counter_offer_id`. Acceptance preserves the proposed branch, while a counteroffer names the admissible change that the requester must incorporate into its revision. Rejection creates no commitment and ends the interaction before a second generation or dispatch. In the controlled workcell diagnostic, exact equality with the prescribed initial proposal gates revision, and the matching proposal joins the delivered response in the next prompt. Table S2 specifies this contract, while environmental state is accessed by execution and verification rather than by the response policy.

Algorithms S1–S2 show how parsing, commitment checks, materialization, and execution compose into a complete interaction. The verifier returns `SUCCESS`, `EXECUTION_FAILURE`, `HANDOFF_FAILURE`, or `TERMINAL_FAILURE`, so an admitted object can be distinguished from the physical outcome of carrying it out.

The algorithms use generation to realize the received work, but deterministic reception replaces that step with validation and instantiation of the same request. During message substitution, admission follows the delivered request while task evaluation keeps the original specification, as detailed in Section S4.2. The GOAL adapter instead searches for a continuation and checks it against the transmitted target and constraints, preserving the distinction between a proposed solution and the outcome it promises.

Algorithm S2 Feedback resolution from the prescribed initialization

Require: requester context x_{req}, peer mode c_{peer}, prior $\mathcal{K}$, public episode instantiation ξ_{pub}, executable world ω_t, prescribed initial proposal $q_0^\star$

$\hat{y}_0 \leftarrow \textsc{Generate}(x_{\text{req}}, \mathcal{K}, \xi_{\text{pub}})$
$q_0 \leftarrow \textsc{Parse}(\hat{y}_0)$
if $q_0 = \bot$ **then**
  return $\textsc{Fail}(\texttt{PARSE_FAILURE})$
if $\neg\textsc{ContractCheck}(q_0, x_{\text{req}}, \mathcal{K}, \xi_{\text{pub}})$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
if $|q_0.\texttt{peer_requests}| \neq 1$ **or** $q_0.\texttt{peer_requests}[0].\texttt{requested_plan} = [\,]$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
if $q_0 \neq q_0^\star$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
$r_{\text{req}\to\text{peer}} \leftarrow q_0.\texttt{peer_requests}[0]$
$\rho \leftarrow \textsc{ResponsePolicy}(r_{\text{req}\to\text{peer}}, c_{\text{peer}}, \mathcal{K}, \xi_{\text{pub}})$
if $\neg\textsc{ResponseValid}(\rho, r_{\text{req}\to\text{peer}}, \mathcal{K}, \xi_{\text{pub}})$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
if $\rho = \textsc{Reject}$ **then**
  return $\textsc{NoCommitment}(\rho.\texttt{reason_code})$
$\hat{y}_1 \leftarrow \textsc{Generate}(x_{\text{req}}, \mathcal{K}, \xi_{\text{pub}}, q_0, \rho)$ ▷ ACCEPT or COUNTER
$q_1 \leftarrow \textsc{Parse}(\hat{y}_1)$
if $q_1 = \bot$ **then**
  return $\textsc{Fail}(\texttt{PARSE_FAILURE})$
if $\neg\textsc{ContractCheck}(q_1, x_{\text{req}}, \mathcal{K}, \xi_{\text{pub}})$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
if $|q_1.\texttt{peer_requests}| \neq 1$ **or** $q_1.\texttt{peer_requests}[0].\texttt{requested_plan} = [\,]$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
if $q_1.\texttt{peer_requests}[0].\texttt{target} \neq r_{\text{req}\to\text{peer}}.\texttt{target}$ **then**
  return $\textsc{Fail}(\texttt{CONTRACT_REJECT})$
$C \leftarrow \textsc{Resolve}_{\leftarrow}(q_0, \rho, q_1, \mathcal{K}, \xi_{\text{pub}})$
if $C = \bot$ **then**
  return $\textsc{Fail}(\texttt{RESOLUTION_CONFLICT})$
$D \leftarrow \textsc{Materialize}(C, \mathcal{K}, \xi_{\text{pub}})$
if $D = \bot$ **then**
  return $\textsc{Fail}(\texttt{MATERIALIZATION_FAILURE})$
$\texttt{status} \leftarrow \textsc{ExecuteAndVerify}(D, \omega_t)$
if $\texttt{status} \neq \texttt{SUCCESS}$ **then**
  return $\textsc{Fail}(\texttt{status})$
return $\textsc{Success}(C, D)$

Table S2. Response options and validation outcomes. Each option identifies the information returned to the requester and the action permitted by the protocol.

Decision / stage	Valid fields or condition	Protocol consequence
Accept	reason code; no `counter_offer_id`	second requester call preserves the normalized initial branch
Counter(α)	reason code and exactly one admissible `counter_offer_id`=α	second requester call realizes the selected bounded branch
Reject	reason code; no counter alternative	creates no commitment; no second generation or dispatch
Invalid response	unknown decision, forbidden field, missing alternative, or inadmissible ID	`CONTRACT_REJECT`
Parse	raw string cannot be decoded	`PARSE_FAILURE`
Contract	schema, role, skill, argument, target, or response check fails	`CONTRACT_REJECT`
Resolution	realization, boundary fields, branch, or dependency merge conflicts	`RESOLUTION_CONFLICT`
Materialization	validated commitment cannot instantiate a dispatch	`MATERIALIZATION_FAILURE`
World discharge	runtime action, inventory handoff, or terminal predicate fails	`EXECUTION_FAILURE`, `HANDOFF_FAILURE`, or `TERMINAL_FAILURE`

S1.1 Resolver Semantics

The resolved object $C = (S_i, S_j, \beta, P_C)$ contains the operative paths, assignment, and ordering that the materializer needs, while the proposal and response identify how the commitment was reached. The codebook identifier b denotes assignment β. As in main Eq. (1), delivery is owed by the sender and completion becomes due after verified receipt [10, 46]. We evaluate fulfillment as $F(C, \xi) = H_\beta(\xi) \wedge T_\beta(\xi)$ and task success as $G_z(\xi) = \text{ExecOK}(\xi) \wedge P_z(\omega_T) \wedge A_z(\xi)$, keeping the original outcome and allocation predicates fixed when a message changes. A receipt remains valid after its materials are consumed, whereas a rejected proposal creates no physical fulfillment observation.

Before comparing paths, $\text{Actorize}(z, a)$ fills every implicit actor in path z with agent a, making the division of work explicit. We then apply $N^{\text{path}}_{\mathcal{K}}$ or $N^{\text{prop}}_{\mathcal{K}}$ to normalize aliases, typed defaults, grounded references, assignments, actors, and predecessor semantics.

Forward resolution. Define

$$S_{\text{snd}} = N^{\text{path}}_{\mathcal{K}}(\text{Actorize}(q_{\text{snd}}.\texttt{self_plan}, \text{snd}))\,,$$

$$R_{\text{rcv}} = N^{\text{path}}_{\mathcal{K}}(\text{Actorize}(r_{\text{snd}\to\text{rcv}}.\texttt{requested_plan}, \text{rcv}))\,, \quad U_{\text{rcv}} = N^{\text{path}}_{\mathcal{K}}(\text{Actorize}(q_{\text{rcv}}.\texttt{self_plan}, \text{rcv}))\,.$$

Under the path contract, the receiver realizes the request exactly when

$$\text{Realizes}_{\mathcal{K}}(U_{\text{rcv}}, R_{\text{rcv}}) \iff U_{\text{rcv}} = R_{\text{rcv}}. \tag{S1}$$

This equality compares the ordered tasks and their skills, values, actors, locations, and dependencies after normalization, so accepted aliases and argument ordering may differ. The realized path contains the requested tasks, while the executor adds its deterministic bookkeeping after resolution.

We write $\text{HandoffCompatible}_{\mathcal{K},\xi_{\text{pub}}}(S_i, S_j)$ for matching of the producing and receiving actors, item, quantity, locations, and dependency under the public task instantiation. The forward resolver admits $C^{\rightarrow} = (S_{\text{snd}}, U_{\text{rcv}}, \beta, P_C)$ when the receiver is the request's target, its `SELF` path is nonempty, its outbound request is empty, Equation S1 holds, and the handoff is compatible. It also requires a complete acyclic merge, preserving each agent's task order and connecting the handoff producer to the receiver's first consuming task. These checks establish an executable commitment, while physical execution establishes receipt. Missing fields, incompatible transfers, multiple selected branches, and cycles return $\perp$.

Feedback resolution. Feedback resolution first determines which branch the response permits. Let $\mathcal{B}_{\mathcal{K}}$ contain the public branches, let B extract a normalized branch from a legal proposal, and let $\text{AltBranch}_{\mathcal{K},\xi_{\text{pub}}}$ map an admissible alternative identifier to its branch. The response selects:

$$B^{\text{resp}}_{\mathcal{K},\xi_{\text{pub}}}(\rho, q_0) = \begin{cases} B(q_0), & \rho = \textsc{Accept}, \\ \text{AltBranch}_{\mathcal{K},\xi_{\text{pub}}}(\alpha), & \rho = \textsc{Counter}(\alpha), \\ \perp, & \rho = \textsc{Reject}. \end{cases}$$

After acceptance, the requester may change surface expression while preserving the initial proposal's normalized branch, peer, and goal. After $\textsc{Counter}(\alpha)$, it preserves the peer and goal but replaces the operative branch with the named alternative. Rejection ends processing without creating a commitment.

For accepted and countered proposals, define

$$S_{\text{req}} = N^{\text{path}}_{\mathcal{K}}(\text{Actorize}(q_1.\texttt{self_plan}, \text{req}))\,,$$

$$R_{\text{peer}} = N^{\text{path}}_{\mathcal{K}}(\text{Actorize}(q_1.\texttt{peer_requests}[0].\texttt{requested_plan}, \text{peer}))\,.$$

$\textsc{Resolve}_{\leftarrow}$ requires

$$B(q_1) = B^{\text{resp}}_{\mathcal{K},\xi_{\text{pub}}}(\rho, q_0),$$

$\text{HandoffCompatible}_{\mathcal{K},\xi_{\text{pub}}}(S_{\text{req}}, R_{\text{peer}})$, and a complete acyclic merge P_C, returning $C^{\leftarrow} = (S_{\text{req}}, R_{\text{peer}}, \beta, P_C)$ when those conditions hold. The materializer uses the resolved paths to construct $D = \textsc{Materialize}(C, \mathcal{K}, \xi_{\text{pub}})$, instantiating task actions, actors, locations, dependencies, and their attached checks. Because these are task-level operations, materialization precedes compilation into navigation, inventory transfers, and other Mineflayer execution parameters.

S2 Commitment Schema and Execution

This section supports the representation and execution mechanism in main Section 4 by giving the complete task grammar and the checks attached to each stage. The runtime example also shows how a compact proposal becomes a longer sequence of grounded skills.

S2.1 Short DSL Grammar

```
program ::= self_line NEWLINE req_line
self_line ::= "SELF -" | "SELF " path
req_line ::= "REQ -" | "REQ " agent " " path
path ::= task (" > " task){0,4}
task ::= skill "(" [args] ")"
args ::= arg ("," arg)*
arg ::= key "=" value
key ::= ident
value ::= json_string | json_number
          | "true" | "false" | "null" | atom
skill ::= control.wait | resource.obtain
          | resource.deliver | craft.item
          | transform.supply_input
          | transform.supply_fuel
          | transform.collect_output
          | build.component
ident ::= [A-Za-z][A-Za-z0-9_]*
agent ::= atom
atom ::= [A-Za-z0-9_.:/+-]+
json_number ::= JSON numeric literal
json_string ::= RFC 8259 double-quoted string
```

We define a canonical grammar while allowing compatible spellings to normalize to the same task object, because the contract concerns the specified work rather than argument order. Values are recognized as quoted JSON strings, complete JSON numbers, reserved literals, and then unquoted ASCII atoms, with escapes decoded before type checking. Quoted strings are scanned atomically, including embedded punctuation and escaped quotes. The serializer uses `q`, `item`, `input`, `to`, `dst`, `bind`, `site`, `from`, `dst_role`, `loc`, and `station`. The parser also canonicalizes the aliases `count`/`quantity`, `target_agent_ref`/`to_agent_id`, `destination`, `destination_role`, `from_agent_id`, `location_ref`, and `binding_id`. Contract checks validate fields and types after parsing, rejecting duplicate keys, unsupported paths, invalid atoms, trailing text, and empty requested continuations.

Each nonempty path contains one to five tasks with positive integer quantities, grounded location references, and an optional public branch identifier. The fields `to` and `bind` name the peer and branch, while `dst`, `site`, `loc`, and `station` supply grounding references. The separator `>` proposes precedence for resolution, and `REQ -` produces an empty `peer_requests` list. Parsing returns $\perp$ for malformed text, whereas a parsed object can subsequently receive `CONTRACT_REJECT`.

We distinguish a syntactically admissible string from an object that satisfies the task contract, since the two checks answer different questions. Let $\mathcal{Y}_{\text{parse}}$ contain admissible strings, $\mathcal{Q}_{\text{parsed}}$ parsed proposals, and $\mathcal{Q}_{\text{legal}}(\mathcal{K}, \xi_{\text{pub}})$ proposals passing the typed contract. Parsing maps into $\mathcal{Q}_{\text{parsed}} \cup \{\perp\}$, giving

$$\mathcal{Y}_{\text{legal}}(\mathcal{K}, \xi_{\text{pub}}) = \{y \in \mathcal{Y}_{\text{parse}} : \text{Parse}(y) \neq \perp,\ \text{ContractCheck}_{\mathcal{K},\xi_{\text{pub}}}(\text{Parse}(y)) = \texttt{true}\}.$$

For a legal string, decoding applies proposal normalization after parsing: $\text{Decode}_{\mathcal{K},\xi_{\text{pub}}}(y) = N^{\text{prop}}_{\mathcal{K}}(\text{Parse}(y))$. We encode legal objects as canonical strings in $\mathcal{Y}_{\text{canon}}(\mathcal{K})$, defining equivalence between two legal strings by

$$y_1 \equiv_{\text{sem}} y_2 \iff \text{Decode}_{\mathcal{K},\xi_{\text{pub}}}(y_1) = \text{Decode}_{\mathcal{K},\xi_{\text{pub}}}(y_2). \tag{S2}$$

We check the following round-trip invariant for every legal proposal Q, which states that encoding and decoding preserve its normalized task meaning:

$$\text{Decode}_{\mathcal{K},\xi_{\text{pub}}}(\text{Encode}_{\mathcal{K}}(Q)) = N^{\text{prop}}_{\mathcal{K}}(Q). \tag{S3}$$

The invariant preserves the complete ordered paths, target peer, typed arguments, actors, grounding references, and dependencies, allowing serialization to change without changing the work resolved by the protocol.

```
SELF resource.obtain(q=8,item=oak_planks) >
     resource.deliver(q=8,item=oak_planks,to=agent_a)
REQ agent_a craft.item(bind=WORK_BRANCH,q=1,
     input=oak_planks,item=crafting_table)
```

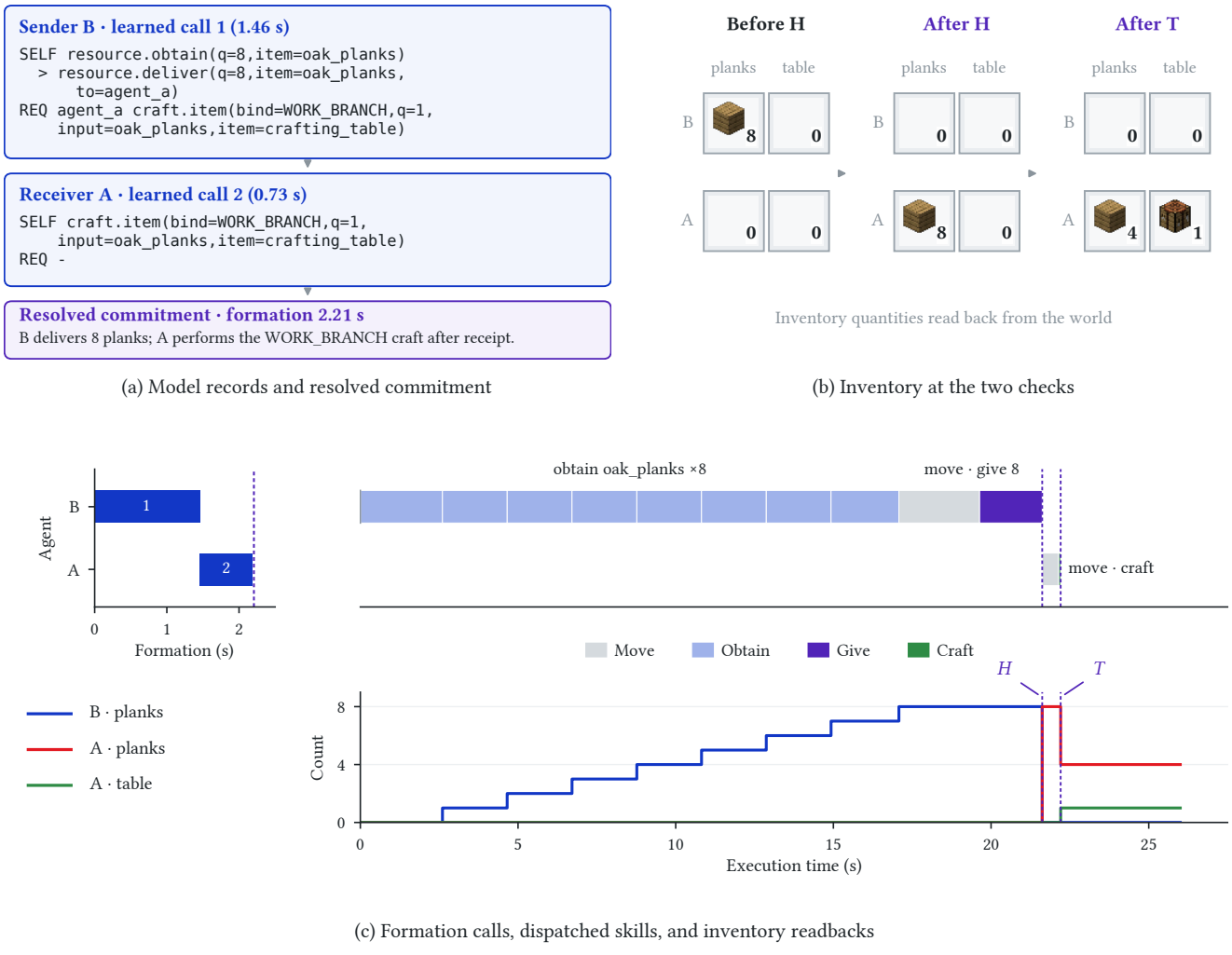
Table S3 connects each validation step to the evidence it establishes, distinguishing commitment checks before dispatch from the observations required for physical fulfillment.

After admission, the materializer combines the selected assignment with the shared task structure, and the compiler produces the corresponding Mineflayer 4.37.1 skills [41]. We execute these skills on Minecraft Java 1.21.4, where inventories and terminal objects provide the readback used to check fulfillment.

Figure S2 illustrates this expansion for an Active Branch task: two model calls produce thirteen dispatched skills and a verified handoff followed by the declared outcome. The episode takes 22.36 s, showing how the time spent forming a compact commitment fits within complete environmental execution.

Table S3. Validation and physical verification through the execution pipeline. The checks establish an admissible commitment before evaluating transfer and completion.

Stage	Check	Accepted condition
Pre-materialization	Syntax and identity	two-line grammar; actors and peer match the role instance
	Skill and arguments	catalog path, required keys, types, and quantities are valid
	Binding and handoff	branch is admissible; item, quantity, sender, and receiver agree
	Dependency merge	selected predecessor graph is complete and acyclic
Post-materialization	Static plan consistency	resolved actors, fields, handoff edge, and checker attachment match the validated commitment
Post-execution	World verification	handoff event and declared terminal predicate both pass



(a) Model records and resolved commitment

(b) Inventory at the two checks

(c) Formation calls, dispatched skills, and inventory readbacks

Figure S2. A complete Short DSL episode. Panel (a) follows sender B's proposal and receiver A's realization to commitment formation at 2.21 s. Panel (b) shows server-read inventories before and after transfer and at completion, omitting tool items. Panel (c) aligns the two generation calls with thirteen dispatched skills and inventory readbacks, with dashed lines locating the handoff H and terminal T checks.

S3 Tasks and Training

This section supports the local-model comparisons in main Section 5 by describing how task assignments become supervision for both participants. We give the task families and training budgets because representation and horizon controls use different prediction targets and numbers of decisions.

S3.1 Tasks and Supervision

Table S4 identifies which part of the requested work changes between the two assignments in each template. Table S5 then connects those assignments to reciprocal training targets for the sender's proposal and the receiver's continuation.

Table S4. The eight task templates and the assignment fields that distinguish their alternatives.

Family	Template	Source material	Balanced bindings	Binding-dependent request path	Changed field
Destination	Build Site A/B	oak planks	`SITE_A/SITE_B`	`build.component`	destination
Destination	Chest A/B	cobblestone	`CHEST_A/CHEST_B`	`resource.deliver`	destination
Recipe	Chest/Crafting Table	oak planks	`CHEST/ CRAFTING_TABLE`	`craft.item`	post-handoff transform; output
Recipe	Planks/Sticks	oak planks	`PLANKS/ STICKS`	`craft.item/ resource.deliver`	post-handoff transform; output
Allocation	Deposit/Build	cobblestone	`BUILD/DEPOSIT`	`build.component/ resource.deliver`	assigned remaining work
Allocation	Dual Build	oak planks	`ROOF/WALL`	`build.component`	assigned component
Active branch	Active Order	oak planks	`STORAGE/WORK`	`craft.item`	downstream continuation
Active branch	Remaining Terminal	oak planks	`DEPOT/ MARKER`	`build.component/ resource.deliver`	downstream continuation

Every template balances two admissible assignments and requires a verified transfer followed by a task-specific outcome. For example, both Recipe templates transfer `oak_planks`, but they change the receiver's subsequent transformation and product. The sender observes the active assignment, whereas the receiver sees a matched context until the request arrives. This construction supports a check of whether the delivered request selects the intended continuation, with the information bound derived in Section S12.1.

Table S5. Reciprocal supervision for the sender proposal and receiver continuation in the forward interaction.

Record stage	Target object	Rows
Sender proposal	multi-step `SELF` plus peer `REQ`	960
Receiver after request	downstream `SELF`; empty `REQ`	960
Intermediate sender (single-step)	next local action after obtain	960

We instantiate supervision from task specifications, sampled local facts, and canonical executable objects, using Minecraft resources to define skills and scenarios. The builder validates every target against its intended assignment and terminal predicate, while keeping the target distinct from the context presented to the model.

S3.2 Training Protocols

Tables S6 and S7 describe the models, stage counts, optimization budgets, and checkpoint rules that produce the compared implementations. The budgets make explicit how each communication or control protocol is supervised.

Table S6. Training examples and optimization updates per checkpoint for each prediction and control protocol.

Model or condition	Rows	Stage counts	Epochs	Steps	Seeds
Multi-step / direct / surface	1,920	960 sender + 960 receiver	2	240	3
Self-plan-only	1,920	960 sender + 960 receiver	2	240	3
Matched-SFT free-form	2,400	1,200 sender + 1,200 receiver	2	300	3
Single-step	2,880	960 sender + 960 intermediate sender + 960 receiver	2	360	3
Factor backend	1,920	960 sender code + 960 receiver code	2	240	3

Table S7. Model configuration and supervision budgets for the horizon comparison.

Field	Multi-step	Single-step
(a) Common configuration		
Base model	Qwen3.5-0.8B [42]	
Optimizer	AdamW; LR 10^{-5}; weight decay 0.01; cosine schedule; warm-up 0.03; grad-norm 1.0	
Batch / precision	micro 4; accumulation 4; effective 16; bfloat16 on A100 40GB	
Length / decoding	maximum 2,048; deterministic decoding; structured cap 256 tokens	
Objective / selection	target-and-EOS causal loss; final-step checkpoint; DEV reserved for implementation checks	
(b) Model-specific training budgets		
Training rows / updates	1,920 / 240	2,880 / 360
Seed IDs	`Seeds 1–3`	`Seeds 1–3`

S4 Evaluation Units and Information

This section supports the comparisons in main Sections 5–6 by identifying the independent evaluation units and the information available when each component acts. These definitions distinguish transfer across tasks, wording, and implementations while keeping their respective outcome measures interpretable.

S4.1 Splits and Statistical Units

Table S8 pairs each comparison with its supplied information and evaluation unit, so repeated roles or checkpoints remain distinguishable from independent task instances.

The horizon test holds out Handoff Station A/B, Smelt Output Binding, Fuel-or-Input Allocation, and Partial-Progress Branch. Syntactic recombination evaluates new expressions through execution, whereas the paraphrase and distractor planning tests measure assignment selection before realization. API repeats sample calls to one service and local repeats sample trained checkpoints, with full online language results reported in Section S9.

Uncertainty. We average repeated roles and checkpoints within the relevant cluster before paired bootstrap inference [17], using 10,000 percentile resamples. Workcell sensitivity additionally pairs the two modes for a fixed goal and world, while source-level language inference averages within each original phrasing. Exact-binomial intervals use two-sided 95% Clopper–Pearson coverage. The prospective comparison resamples 640 descriptions within 16 choices and uses 97.5% intervals for its two primary effects, with 95% intervals for secondary contrasts. Sections S10.2 and S9 explain how source stratification changes the language estimand.

S4.2 Information Access and Task Predicates

Table S9 distinguishes the inputs available to a model from the references used by validation and evaluation, making the timing of each information use explicit.

Workcell Revision Gate. In the controlled workcell diagnostic, we invoke revision when the parsed initial proposal exactly matches the prescribed object, which then enters the history as `INITIAL_PROPOSAL_DELIVERED`. The second prompt combines that history with the delivered response and local context, while the final reference is applied after generation. Initial failures remain unsuccessful assigned trials.

Delivered content and evaluation references. Forward evaluation preserves the original proposal, delivered request, receiver output, and admitted object because they serve different purposes. Request consistency can reject a syntactically legal alternative that contradicts the delivered request, as the examples in Section S12.5 illustrate. The runtime executes the admitted assignment, while task evaluation continues to use the original requirement, allowing discrepancies to be localized to generation, admission, or environmental execution.

Workcell task predicates. The workcell mode assigns the crafting actor and the item that must be handed over, although either agent can physically craft the product. We require exactly one verified handoff, one task-relevant craft by the assigned actor, and the specified product and count at the destination. As a result, the two processing modes have disjoint acceptable routes even when both produce the same terminal item, which supplies the paired-information diagnostic of Section S12.1.

Forward task predicates. The forward templates change destinations, products after transfer, storage or construction choices, assigned wall or roof components, or downstream terminals. We compare the admitted assignment with the original task and verify its physical consequences, using a shared candidate graph in the core tests. In the private-structure tests, the message supplies the continuation that the receiver combines with its shared primitives.

Table S8. Evaluation design by comparison. Each row states what changes, which information remains available, and which units support inference.

Pool	Changed or held out	Retained information	Counts / unit
Core requests	New instances within eight templates	Same skill ontology, task schema and public candidates	160 semantic clusters; roles and three seeds nested
Template horizon	Four whole templates absent from training	Same four families and skill ontology; task structure supplied	80 clusters, 480 evaluations per learned method
Surface/backend	Separately sampled instances, within known template families	Same shared task structure and execution path	128 quality clusters; independent 40-cluster timing samples
Workcell feedback	New instances of the four goal–mode cells	Same two goals and two modes; prescribed initial proposal	40 clusters; 20 paired mode blocks
Description wording	Lexical roots and paired distractors	Sixteen semantic choices seen in training	32 shared original phrasings; 320 role rows per condition
API request transfer	New paired blocks and another formation backend	Public task structure and four worked examples	80 pairs; 960 assignments per backend/condition
Receiver comparison	Receiver realization implementation	Same 1,920 actual sender outputs	Formation replay; seven additional physical objects; timing on 16 blocks
Private structure	Sender keeps task DAG; parameterized instances	Shared primitives and explicit transmitted boundary	114 parameter specifications within 24 families; 96-spec sender probe
Stage–order	Presentation of the same graph nodes	Same 96 source clusters, targets and checkpoints	Four paired views; 576 evaluations each
Online descriptions	Full formation, wire, and Minecraft execution	Same language sources and shared executable interface	10,240 quality trials on 32 phrasings; 960 timed trials on 16 phrasings
Prospective selection	New descriptions and score-based routing	Sixteen task choices, fixed selectors and receiver	640 test descriptions; 17,920 quality trials; 1,024 timed trials on 64 descriptions

S5 Communication and Commitment Horizon

This section supports main Sections 6.1–6.2 by comparing communication costs at matched task success and the effects of a longer executable commitment. Complete distributions and family-level results complement the aggregate comparisons in the main paper.

S5.1 Communication Surfaces

Table S10 compares the three surfaces at matched observed quality, while Table S11 gives call latency and the reliability of the full quality pool. Together they distinguish communication cost from the execution shared by the protocols.

The JSON control is trained to emit `self_plan` and `peer_requests`, preserving descriptive nested keys such as `task_path`, `arguments`, `requested_plan`, and `depends_on`. Its training targets use sorted keys on one line with no optional whitespace. Short DSL expresses the corresponding tasks as ordered skill paths and argument lists, using aliases such as `q` and `to`. We compare generated outputs and measured TTC for these concrete encodings.

The surface timing sample crosses 40 clusters with two roles and three checkpoints per surface, running resident models sequentially on the same device. Its instances are distinct from the backend sample in Section S8.1. TTC ends when the bilateral commitment is available, while the subsequent executor time stays near a 22.6 s median across surfaces.

Table S12 reports uncertainty for the communication-cost contrasts using paired task clusters. The local comparison uses 40 clusters, and the 64 interleaved DeepSeek pairs span 29 clusters; the intervals use 10,000 bootstrap draws.

Table S9. Information available when a component forms, validates, or evaluates work. The columns distinguish decision inputs from validation and task references.

Component	Decision input	Reference or implementation boundary
Forward sender	Sender-local fact and shared task structure	Model input contains the local task fact; evaluation uses the target after generation
Forward receiver	Local post-handoff context and delivered request	The expected object is built from the delivered request; raw-output consistency is checked before admission
Workcell initial requester	Private goal and public alternatives	Peer mode is absent; exact match to the prescribed initial object gates revision
Workcell revision	Local context, initial-proposal history, delivered response	History stores the canonical initial object only after equality with the actual parsed initial output
Deterministic response policy	Requested suffix and peer-local processing mode	Deterministic finite alternatives; no language-model call
Core materializer	Admitted binding and shared task structure	Instantiates task actions and dependencies from the admitted binding and shared graph
Private-structure receiver	Shared primitive semantics, local grounding, actual message	No private task DAG or hidden task-specific candidate table
World evaluator	Actual trace, original goal/mode, declared obligation	Scores task success and fulfillment from the fixed specification and execution trace
Naturalized lexical baselines	Local text and public template identifier	TRAIN labels constrain retrieval/classification; the oracle rule is a different reference
Centralized quality reference	Both current local views and shared prior	Quality reference with complete task information

Table S10. Communication surfaces at matched task success. Quality uses 128 clusters, while calls, tokens, peer payload, and TTC use the independent 40-cluster surface timing sample.

Method	Success	Calls	Tokens		Wire B	TTC (s)	
			Input	Output		p50	p95
Short DSL GenCoord (0.8B)	100.0%	2.0	949.1	78.6	76.3	2.27	2.62
JSON GenCoord (0.8B)	100.0%	2.0	1,154.7	267.6	303.4	7.07	7.86
Controlled free-form +structured commit (0.8B)	100.0%	2.5	1,585.2	265.8	1,061.1	7.13	7.86

Table S11. Reliability and call latency for each communication surface. Reliability uses the 128-cluster quality pool and latency uses the 40-cluster timing sample.

(a) Call latency: 40-cluster sample

Surface	p50 (s)	p95 (s)
Short DSL	1.14	1.66
JSON	3.53	5.94
Controlled free-form	3.27	3.75

(b) Reliability: 128-cluster quality pool

Measure	Value
Strict successful clusters, each surface	128/128
Exact-binomial 95% lower bound	97.2%
Role-swap success difference	0.0 pp

Table S12. Cost contrasts relative to Short DSL. Reductions compare Short DSL with the natural-language protocol; ratios divide the named protocol by Short DSL.

Backend	Contrast	Estimate	95% CI
Local 0.8B	Peer-traffic reduction (%)	92.8	[92.7, 93.0]
	Median TTC reduction (%)	68.2	[67.9, 68.7]
	JSON output-token ratio	3.40	[3.37, 3.44]
	Free-form output-token ratio	3.38	[3.34, 3.42]
	JSON median-TTC ratio	3.12	[3.08, 3.16]
	Free-form median-TTC ratio	3.15	[3.11, 3.20]
DeepSeek	Mean TTC reduction (%)	30.4	[22.9, 38.6]
	Median TTC reduction (%)	20.8	[14.4, 32.9]

S5.2 Commitment Horizon

Table S13 reports the aggregate paired effects and their family decomposition, which explain the success and decision differences plotted in main Figure 4(c,d).

Table S13. Multistep versus single-step control on 80 held-out-template clusters. Differences are multistep minus single-step, except the final rule comparison. Success differences use percentage points and decision/token differences use counts per episode, with intervals for aggregate paired effects.

Family / metric	Multi-step	Single-step	Δ	95% CI
Destination	100.0%	100.0%	0.0	—
Recipe	100.0%	100.0%	0.0	—
Allocation	92.5%	83.3%	9.2	—
Active branch	100.0%	81.7%	18.3	—
Overall success	98.1%	91.3%	6.9	[2.9, 10.8]
Decisions / episode	1.98	2.91	−0.93	[−0.97, −0.89]
Input tokens / episode	1,146.5	1,689.1	−542.6	[−567.8, −517.6]
Multi-step versus rule	98.1%	100.0%	−1.9	[−4.2, −0.2]

Multistep control uses 1,920 examples and 240 updates per seed, whereas single-step control receives 2,880 examples, 360 updates, and explicit supervision for an intermediate decision after acquisition. We compare those complete training and control protocols because the additional decision changes both the available supervision and online interaction. The experiment extends communicated multistep intent [5] to executable obligations whose duration determines when another decision is needed.

The deterministic correct-assignment rule reaches the held-out task ceiling in Table S13, while Direct preserves the requested assignment in all 471 executable outputs. Its nine remaining trials stop before executable-plan formation, localizing the difference to producing the object rather than carrying out an admitted continuation.

S6 Prompted Communication and Revision

This section supports main Sections 6.1–6.2 by comparing the two communication protocols with the same DeepSeek model and by evaluating revision after a partner supplies structured feedback. We describe the dialogue contract before reporting quality, model usage, and the independent timing sample, so the comparison can be interpreted as two ways to form the same kind of commitment.

S6.1 Messages, Examples, and Acceptance

Both protocols use DeepSeek-V4.1-Flash for the two participants, the same task instances, and the same skill vocabulary, commitment checks, and world executor. Each prompt contains four worked examples, two for each role orientation, clearly separated from the current instance. The dialogue examples are the teacher conversations used to train the local controlled free-form system. All are initiated by the material supplier, and the current prompt identifies the agent's and partner's public roles. Short DSL likewise uses role-balanced examples with instructions appropriate to the agent's current role. The demonstrations and serialized task context enter the input-token counts alongside the interaction history. These prompt components are specific to each protocol, so input totals include both the examples that teach the protocol and the messages exchanged while solving the task.

The dialogue protocol allows natural-language discussion accompanied by structured control markers, with at most eight logical model calls. A `COMMIT` marker encloses the two agents and their proposed task lists, while a valid `ACCEPT_COMMIT` accepts a referenced proposal. The partner can also close the commitment by submitting the matching valid object. The controller validates the structured object and acceptance relationship. Explanatory text can discuss conditions or alternatives without changing a valid control marker, whereas an explicit `<REJECT/>` rejects the proposal. Thus the two agents may explain their work in natural language while the markers identify precisely what they submit and accept.

For example, a supplier can state which branch the order requires, promise to obtain and deliver its material, and include the corresponding structured proposal in the same turn. The receiver then accepts the proposal or submits the matching object before the controller closes the commitment. The complete example dialogues in the supplementary data show both forms of acceptance. Short DSL instead expresses the sender's work and requested continuation directly as skill paths, which the path resolver checks under the protocol in Section S1.

S6.2 Task and Formation Outcomes

We evaluate 128 task clusters with two role assignments and three API repetitions per protocol, giving 768 assigned trials in each arm. Table S14 distinguishes commitment formation from handoff, fulfillment, and success on the original task, while Table S15 breaks these results down by role orientation.

Table S14. Task outcomes with the same prompted DeepSeek model. C counts formed commitments, H verified handoffs, F fulfilled commitments, and G successes on the original task. Counts use all assigned trials.

Protocol	Assigned	C	H	F	G
Short DSL	768	768	768	768	768
Dialogue + structured submission	768	767	767	767	766

Table S15. Task outcomes by role orientation. Original and swapped roles each contain 384 assigned trials per protocol.

Protocol	Roles	Assigned	Formed	Fulfilled	Task success
Short DSL	Original	384	384	384	384
Short DSL	Swapped	384	384	384	384
Dialogue	Original	384	384	384	383
Dialogue	Swapped	384	383	383	383

The dialogue protocol has one unsuccessful trial in each role orientation. One dialogue trial stops before commitment formation, and the other fulfills a commitment that selects work inconsistent with the task. The first proposal in each of these trials assigns the roles correctly. The communication-cost comparison therefore concerns two protocols that both complete almost all tasks, with the stages in Table S14 identifying the two dialogue outcomes.

S6.3 Peer Traffic, Model Usage, and Charges

Table S16 reports per-trial model usage and peer traffic in the quality pool. Peer traffic counts UTF-8 bytes actually delivered between the agents, including commitment and closing control messages. It is therefore distinct from API request bodies and from the model's output-token count. The 768 Short DSL trials use 86,592 peer bytes and 1,536 logical calls, whereas the dialogue trials use 701,770 bytes and 1,631 calls. Although both protocols average close to two calls, dialogue generates more output and transmits more bytes to the partner. Input-token counts also include the protocol-specific examples and context encoding described above.

Table S16. Mean communication and model costs per assigned quality trial. Dialogue token and charge means use 767 known usage entries; other means use 768 entries. Input includes worked examples, context encoding, and interaction history, and charges price both input and output under the stated tariff.

Metric	Short DSL	Dialogue	Dialogue / DSL
Logical calls	2.00	2.12	1.06
Peer traffic (B)	112.75	913.76	8.10
Input tokens	5,573	21,086	3.78
Output tokens	100.4	276.3	2.75
Estimated charge (USD)	0.000284	0.000735	2.59

API charges price the complete prompting configuration, including protocol-specific examples, context encoding, generated responses, and cache use. We estimate them from usage at fixed prices of USD 0.30 per million uncached input tokens, USD 0.006 per million cached input tokens, and USD 1.20 per million output tokens. The charge summaries include usage associated with unsuccessful trials as well as successful ones. One successful dialogue trial has no complete usage estimate, so the table reports means over available usage rather than assigning it zero cost. These estimates represent API charges under the stated tariff, distinct from a supplier invoice or the cost of local computation.

S6.4 Interleaved Timing

The timing comparison uses 64 fresh trials per protocol, interleaving the protocols in a fixed alternating order with HTTP concurrency one and one world at a time. TTC begins with forming the proposal and ends when a commitment is available or the attempt stops, including API service and network time. All 128 timed trials form commitments, although a subsequent Short DSL handoff failure makes that arm's task-success count 63/64. We include all 64 commitment times in its summary because the handoff occurs after formation.

As Table S17 shows, Short DSL reduces mean TTC by 30.4% and median TTC by 20.8%. The timing sample has complete usage estimates for both protocols, with its own prompt and response lengths, so its charge means are reported independently of the larger quality pool. These measurements quantify the delay to forming the commitment and distinguish it from the physical work that follows.

Table S17. Formation time and estimated charges in the fresh timing sample. Each protocol has 64 assigned and formed trials. Times are seconds, and charges are USD per trial. Median averages the central observations; p95 uses the lower empirical 95th-percentile order statistic.

Protocol	Task success	TTC mean	Median	p95	Calls	API charge
Short DSL	63/64	4.29	4.38	5.87	2.00	0.000257
Dialogue	64/64	6.17	5.52	10.03	2.11	0.000599

S6.5 Revising from Structured Feedback

We evaluate feedback when the requester has a private goal and its peer has a private processing mode. In the controlled condition, we supply an initial proposal and either the correct response or a legal alternative. In the full interaction, the requester generates the initial proposal and then revises it after feedback. The response policy and commitment checks are the same in both cases.

The four worked feedback examples balance role orientations, with the peer-receives-finished mode paired with the original roles and the peer-processes-material mode paired with swapped roles. We cross 40 independent mode pairs with two modes, two roles, and three API repetitions, yielding 480 trials per condition. Table S18 shows how formation and physical outcomes relate, while Table S19 gives the role breakdown. Correct-feedback revision succeeds in 87.3% of trials and the full interaction succeeds in 86.3%, demonstrating that the model can revise the actor and handoff from structured feedback.

Table S18. Requester revision with the prompted DeepSeek model. C, H, and F measure formation, handoff, and fulfillment; G measures success under the unchanged original task.

Condition	Assigned	C	H	F	G
Correct response, initial proposal supplied	480	419	419	419	419
Alternative response, initial proposal supplied	480	403	403	403	0
Initial proposal and revision generated	480	414	414	414	414

Table S19. Feedback outcomes by role orientation. Each entry uses 240 assigned trials. The paired effect compares correct with alternative feedback over 40 mode pairs.

	Original roles		Swapped roles	
Condition	Formed	Task success	Formed	Task success
Correct response	208	208	211	211
Alternative response	208	0	195	0
Full interaction	202	202	212	212

The correct-minus-alternative task-success effect is 87.3 percentage points, with a 95% paired bootstrap interval of [84.4, 90.0] from 10,000 resamples of the 40 mode pairs. Alternative feedback produces 403 admitted and fulfilled commitments but no success on the original task, because these commitments follow the substituted division of work. The condition therefore checks how feedback changes the actor and transfer while keeping the task's requirements fixed.

S7 Fulfillment under Private Task Structure

This section supports main Section 6.3 by testing whether a receiver can realize transmitted work while the sender keeps the instance graph private. We report complete execution outcomes alongside message and computation costs for paths, graph fragments, and goals.

S7.1 Cases and Execution Objects

We use deterministic projection to supply a path, graph slice, or goal with constraints, while giving the receiver shared primitives and local grounding. PATH supplies the ordered continuation and SLICE supplies its dependency graph, whereas GOAL searches for work satisfying the transmitted target. Because the original graph stays with the sender, these tests isolate the information the receiving interface needs to act.

We specify 114 parameter settings within 24 template–motif families and cross two versions with two roles, yielding 456 base executions. These are specified cases within the family structure rather than independent task families. We evaluate sender generation on 96 source settings and 384 version–role combinations with the three unchanged Direct checkpoints, complementing the receiver test with a measurement of object formation.

Table S20 reports admission, transfer, fulfillment, and original-task success for each transmitted representation and replacement condition. The separate columns show whether an executable alternative was carried out and whether it matched the original assignment.

Table S20. Physical outcomes when the sender keeps its task graph private. Compatible replacements can fulfill their delivered commitment while leaving the unchanged original task unsatisfied.

Condition	Assigned	Executed	H	F	G
Base PATH	456	456	456	456	456
GOAL with a different legal path	100	100	100	100	100
Compatible alternative request	16	16	16	16	0
Incompatible alternative request	16	0	–	–	–

The 588 assignments include 572 physically fulfilled objects and 16 requests rejected before execution. We select the 100 GOAL continuations because their legal paths differ from the supplied PATH, demonstrating fulfillment through another solution to the declared objective. The original task stays unchanged in both replacement conditions, so the compatible alternatives can be fulfilled even when task success is zero.

S7.2 Cold, Warm, and Changed-Content Accounting

We count a session's application request and acknowledgment, including the method, roles, sequence, and either the task content or a cache reference. A first delivery installs the content, a cache hit reuses it, and changed content is transmitted again. Primitive semantics are shared beforehand, while the ledger measures the task communication required within that session.

Table S21 gives request, acknowledgment, and total transmitted bytes for each cache condition, with stored receiver content shown in its own column. We weight the 114 settings equally after pairing versions and roles, and provide family-balanced summaries in the accompanying data.

Table S21. Transmitted bytes for paths, slices, and goals under first, cached, and changed delivery. Requests include content or a cache reference, every acknowledgment is 135 B, and receiver cache is stored content rather than transmission. With caching off, Any denotes identical costs across the three delivery conditions.

Cache	Delivery	Message	Request (B)	ACK (B)	Total (B)	Cache (B)
Off	Any	PATH	605.63	135.00	740.63	0.00
Off	Any	SLICE	670.30	135.00	805.30	0.00
Off	Any	GOAL	526.89	135.00	661.89	0.00
On	Cold	PATH	605.63	135.00	740.63	348.88
On	Cold	SLICE	670.30	135.00	805.30	401.21
On	Cold	GOAL	526.89	135.00	661.89	273.31
On	Warm	PATH	201.00	135.00	336.00	348.88
On	Warm	SLICE	202.00	135.00	337.00	401.21
On	Warm	GOAL	212.00	135.00	347.00	273.31
On	Changed	PATH	605.63	135.00	740.63	697.75
On	Changed	SLICE	670.30	135.00	805.30	802.42
On	Changed	GOAL	526.89	135.00	661.89	546.61

Table S22. Delivery-processing CPU time in microseconds (μs), including serialization, decoding, realization, checks, and acknowledgment. Entries are medians of setting-level medians after pairing roles and versions, using five technical repeats and excluding model calls and world execution.

Cache	Delivery	PATH	SLICE	GOAL
Off	Cold	65	66	2010
Off	Warm	60	61	2026
Off	Changed	60	62	1983
On	Cold	58	60	1984
On	Warm	50	52	1981
On	Changed	58	60	1989

GOAL saves bytes on first delivery by leaving search to the receiver, whereas PATH and SLICE communicate the task structure that it must realize. Caching preserves this difference because a cached target still requires search, while a cached path already specifies the continuation. Tables S21 and S22 therefore describe both communication and computation within the same application-level exchange.

S7.3 Direct Sender Formation under Private Structure

For the generative probe, we supply the selected route to the unchanged Direct model and ask it to produce the complete sender object. Across 96 source settings, two versions, two roles, and three checkpoints, all 1,152 outputs parse and 496 satisfy the complete-object criterion. Matched executable objects identify the corresponding physical receipts, while this probe measures generation under the new input contract.

Table S23 breaks sender formation down by checkpoint, showing how the 496 complete objects are distributed across the three trained models.

Table S23. Complete sender objects generated by each unchanged Direct checkpoint under private task structure.

Checkpoint seed	Calls	Strictly formed	Formation (%)
Seed 1	384	223	58.1
Seed 2	384	145	37.8
Seed 3	384	128	33.3
All	1,152	496	43.1

The physical interface test establishes what a receiver can execute from a supplied object, whereas the Direct probe measures how reliably a particular sender produces that object. Reporting both makes the role of deterministic projection explicit.

S8 Formation and Receiver Implementations

This section supports the implementation choices in main Section 6.3 by comparing the cost of expressing an assignment and realizing it at the receiver. We first hold the peer request fixed across formation backends, then hold sender outputs fixed across receiver implementations.

S8.1 Backend Formation and Composition

Table S24 compares formation with the same peer-facing object, using a timing sample independent of the surface comparison in Section S5.1. Table S25 examines how the composer combines fields on unseen assignment cross-products, distinguishing that construction from lookup of a complete known case.

Table S24. Direct and Factor formation at matched quality. Success uses 128 clusters, with two roles and three checkpoints, while generation cost and TTC use a separate 40-cluster timing sample.

Metric	Factor + rule	Direct DSL
Held-out success	768/768	768/768
Input tokens	980.2	948.9
Output tokens	19.4	79.1
Protocol-specific peer payload	76.6 B	76.6 B
TTC p50	0.82 s	2.45 s
TTC p95	1.00 s	2.72 s

Factor predicts the sender's assignment code before delivery and the receiver's accepted assignment code afterward, with the same input information available to Direct. A deterministic composer then expands the code into actors, tasks, objects, destinations, dependencies, and the peer-facing Short DSL request.

Table S25. Complete-case lookup versus field-wise composition on 60 unseen assignment cross-products.

Method	Task success
Complete-case table	0/60
Explicit-factor composer	60/60
Direct Short DSL	60/60

Together, Tables S24 and S25 connect a shorter generation target to the complete object produced by the composer. Field-wise composition supports new combinations while preserving the executable request that the partner receives.

S8.2 Fixed-Sender Receiver Realization

We hold 1,920 saved sender outputs fixed, with 960 from each backend, and compare learned reception, deterministic request realization, and shared-directory lookup. Each implementation uses the sender's selected assignment, giving 5,760 formation trials without substituting a correct reference choice. This isolates reception from sender accuracy, while Section S9 evaluates the complete online chain.

Table S26. Learned, deterministic, and directory-based receivers applied to the same saved sender outputs.

Sender source	Learned	Deterministic path	Directory lookup
API (960)	952	959	959
Local (960)	958	958	958
All (1,920)	1,910	1,917	1,917

The seven additional commitments produced by deterministic reception all arise from API sender outputs, and each succeeds in a newly executed world. Directory lookup produces the same seven objects, while the remaining entries in Table S26 compare realization of the saved proposals.

The receiver-stage benchmark crosses 16 world blocks, 64 role instances, three checkpoints, and five technical repeats, producing 960 operations per implementation on one RTX 4090. Timing begins when the receiver processes the request and ends after realization checks, leaving sender generation, loading, and world execution outside the measured stage.

Table S27 reports each implementation's latency distribution, allowing its typical receiver cost to be compared with the paired mean differences below.

Table S27. Receiver processing latency on the common local sample. Each method has 960 operations over 16 world blocks, including all technical repetitions and the long learned-receiver observation.

Implementation	Median (ms)	Mean (ms)	p95 (ms)	Maximum (ms)
Learned	856.11	880.84	972.99	17482.75
Deterministic path	0.63	0.63	0.65	1.21
Directory lookup	0.36	0.36	0.38	0.73

Deterministic realization saves a paired mean of 880 ms [862, 898] relative to learned reception, and directory lookup saves 880 ms [862, 899]. We average within world block before 10,000 bootstrap resamples within template, so the intervals measure changes in receiver processing. The full online measurements in Section S9 additionally account for sender formation and physical execution.

Table S28 reports receiver-stage means for each local checkpoint, showing how the aggregate difference relates to the three trained implementations.

Table S28. Mean receiver processing time in milliseconds by local checkpoint, with 320 operations per method and checkpoint.

Seed	Learned	Deterministic path	Directory lookup
Seed 1	916.77	0.63	0.36
Seed 2	872.80	0.63	0.36
Seed 3	852.94	0.62	0.36

S9 Naturalized Descriptions in Closed-Loop Execution

This section supports the receiver and language comparisons in main Section 6.3 by following natural descriptions through formation, transmission, and actual Minecraft execution. The stage breakdown and paired timing identify which components change when the same assignment interface uses another selector or receiver.

S9.1 Descriptions, Selectors, and Execution

We use 32 original phrasings distributed over 16 template–assignment choices, with paraphrase and distractor conditions derived from the same sources. Each condition has 320 role instances, and the local generators use three checkpoints. We therefore evaluate 10,240 quality trials across the eight configurations in Table S29, keeping the shared language sources explicit in the comparison.

The configurations compare Direct and Factor with learned or deterministic receivers, alongside TF–IDF retrieval, supervised lexical classification, embedding retrieval, and a privileged quality reference. The embedding model has general pretraining, whereas the classifier is fitted on the training descriptions, allowing the interface to support practically different approaches to selecting the same assignment.

Embedding retrieval uses Qwen3-Embedding-0.6B with a fixed index of 64 training descriptions [61], selecting within the public template. The supervised lexical model chooses its regularization on development data, and the local generators use their existing checkpoints with deterministic decoding. These choices preserve each implementation's trained behavior while changing the formation component used by the common receiver.

Table S29. Assignment and receiver implementations for natural descriptions. Generation produces a path or code, while an embedding call encodes the query description. The privileged reference supplies the correct assignment for quality evaluation.

Configuration	Sender formation	Receiver
Direct / learned	Path generation	Learned path generation
Direct / deterministic	Same Direct sender output	Deterministic interpretation
Factor / learned	Short-code prediction and composition	Learned code and composition
Factor / deterministic	Same Factor sender output	Deterministic interpretation
TF–IDF retrieval	Nearest TRAIN description, then composition	Deterministic interpretation
Supervised lexical classifier	TRAIN-fitted logistic regression, then composition	Deterministic interpretation
Embedding retrieval	Pretrained encoder retrieval, then composition	Deterministic interpretation
Privileged true-binding reference	True binding and composition	Deterministic interpretation

The receiver sees `PROJECTED_HANDOFF_CONTEXT`, which specifies the inventory and history expected after the proposed transfer. Once the full commitment has formed, the agents actually execute the handoff and read back the world state, so the online chain validates those projected preconditions physically.

We denote request matching by M, which checks the raw receiver continuation against the delivered request before admission. Physical fulfillment F is evaluated later from transfer and completion, allowing a matched continuation to be distinguished from successful environmental execution. The stage columns in Table S30 preserve this distinction.

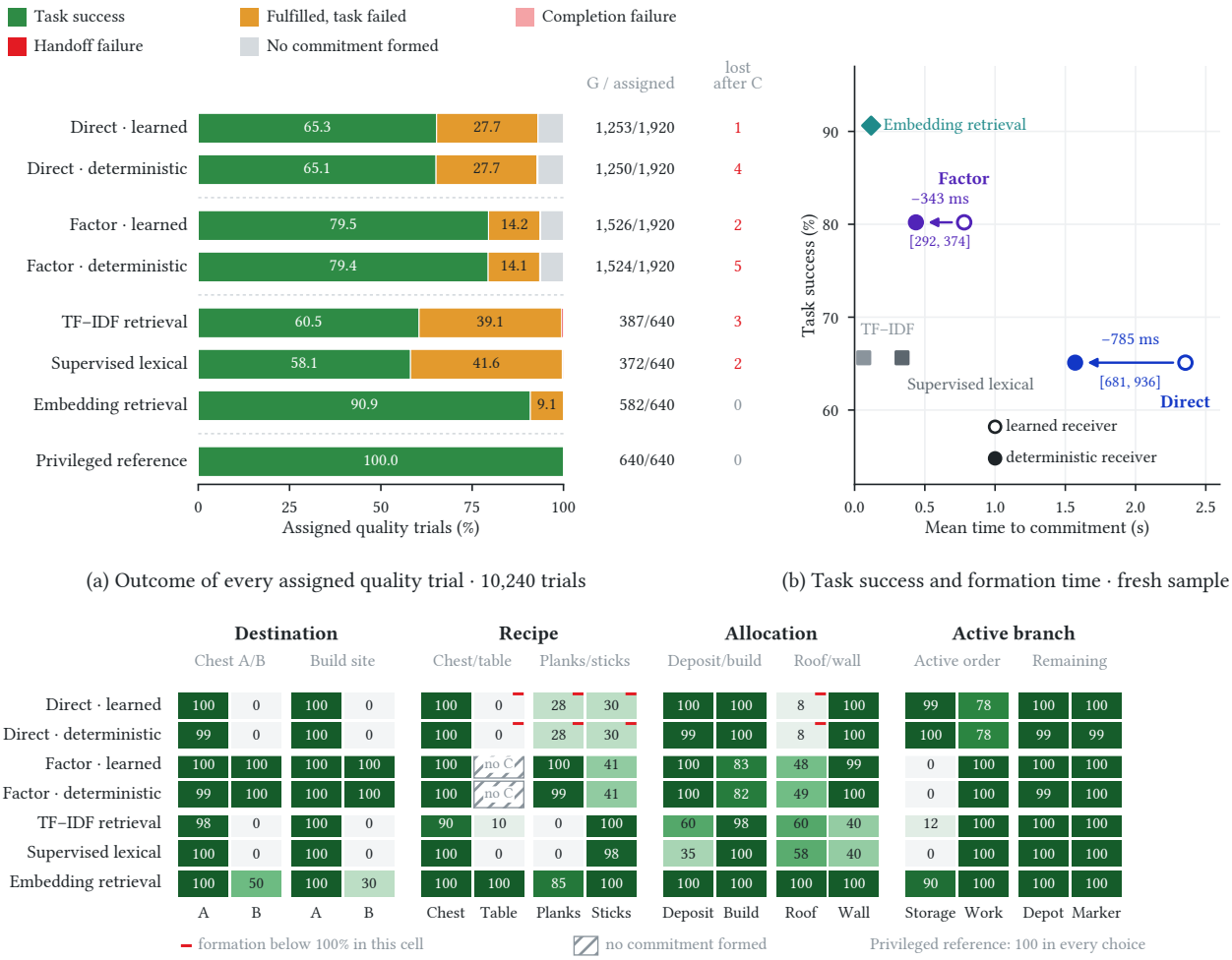
S9.2 From Formation to Physical Outcomes

Figure S3(a) decomposes all 10,240 trials into mutually exclusive outcomes: 508 formation failures, 12 handoff failures, five failures to complete the declared work, 2,181 fulfilled but inappropriate obligations, and 7,534 successful tasks. Thus 9,715 of the 9,732 formed commitments are fulfilled, with the remaining losses occurring during execution.

Table S30. Physical outcomes under each description condition. All assigned trials contribute to C (formation), H (verified handoff), F (declared fulfillment), and G (original-task success), with task success also shown as a percentage.

Configuration	Assigned	C	H	F	G	G (%)
Paraphrases						
Direct / learned	960	898	898	898	632	65.8
Direct / deterministic	960	898	898	898	632	65.8
Factor / learned	960	900	899	899	758	79.0
Factor / deterministic	960	900	900	899	759	79.1
TF–IDF retrieval	320	320	319	319	189	59.1
Supervised lexical classifier	320	320	320	320	190	59.4
Embedding retrieval	320	320	320	320	290	90.6
Privileged true-binding reference	320	320	320	320	320	100.0
With distractors						
Direct / learned	960	888	887	887	621	64.7
Direct / deterministic	960	888	886	884	618	64.4
Factor / learned	960	900	900	899	768	80.0
Factor / deterministic	960	900	897	896	765	79.7
TF–IDF retrieval	320	320	318	318	198	61.9
Supervised lexical classifier	320	320	318	318	182	56.9
Embedding retrieval	320	320	320	320	292	91.3
Privileged true-binding reference	320	320	320	320	320	100.0

Among the 2,198 task failures after formation, 2,181 follow successful fulfillment of an inappropriate obligation, or 99.2%. Embedding retrieval provides a clear instance: all 640 commitments are fulfilled, while the 58 unsuccessful tasks select the



(a) Outcome of every assigned quality trial · 10,240 trials

(b) Task success and formation time · fresh sample

(c) Task success (%) by semantic choice · two original phrasings per choice

Figure S3. From natural descriptions to world outcomes. Panel (a) shows mutually exclusive results over 10,240 assigned trials, normalizing each configuration by its own denominator and reporting task successes and losses after formation at right. Panel (b) compares success and mean TTC on the same fresh timing sample, with arrows and paired 95% intervals for receiver replacement over 16 sources. The lexical mean includes the 17.3 s observation, while its median is 0.06 s. Panel (c) reports success for 16 choices, each derived from two phrasings: hatching marks no formed commitment and red ticks mark formation below 100%. The privileged reference completes every choice and has no timing arm.

wrong assignment. The pattern localizes the main error on these descriptions to selection, while the stage breakdown also exposes the 17 execution losses.

S9.3 Paired Receiver and Selector Comparisons

We pair 3,840 sender outputs across the two receiver implementations, yielding 7,680 quality trials, and add 384 pairs in the fresh timing sample. Because each pair uses the same sender output, both receivers form the same number of commitments and their small task-success differences arise after formation. Table S31 gives the paired outcomes by language condition and selector.

For quality inference, we first average roles, worlds, and checkpoints within each original phrasing, then resample the two phrasings within each semantic choice. An unstratified sensitivity instead resamples all 32 phrasings, allowing the choice mixture to vary. Both use 10,000 paired percentile resamples, and the broader unstratified intervals include zero for Factor versus learned Direct and Factor versus embedding retrieval. Table S32 reports the corresponding changes in formation and fulfillment.

S9.4 Fresh Timing and Online Work

We time one phrasing per semantic choice, giving 16 sources and 64 role instances across the two language conditions. Direct and Factor contribute 192 trials per receiver and each remaining configuration contributes 64, for 960 sequential executions on an exclusively used RTX 4090D. Models are resident and batch size is one. We start timing after world readiness and include

Table S31. Paired differences in task success, in percentage points, for each selector and receiver configuration. Stratified and unstratified intervals use the same source pairing.

Contrast	Difference	Stratified 95% CI	Unstratified 95% CI
Paraphrases			
Factor: deterministic − learned	+0.1	[0.0, 0.2]	[0.0, 0.3]
Direct: deterministic − learned	0.0	[0.0, 0.0]	[0.0, 0.0]
Factor deterministic − Direct learned	+13.2	[10.4, 16.3]	[−3.0, 29.7]
Factor deterministic − TF–IDF	+20.0	[12.0, 28.0]	[2.5, 38.1]
Factor deterministic − supervised lexical	+19.7	[12.0, 27.4]	[2.2, 38.0]
Factor deterministic − embedding	−11.6	[−15.7, −7.4]	[−28.9, 5.8]
With distractors			
Factor: deterministic − learned	−0.3	[−0.6, 0.0]	[−0.7, 0.1]
Direct: deterministic − learned	−0.3	[−0.6, 0.0]	[−0.7, 0.1]
Factor deterministic − Direct learned	+15.0	[12.6, 17.4]	[−1.1, 31.0]
Factor deterministic − TF–IDF	+17.8	[11.4, 24.3]	[0.7, 35.4]
Factor deterministic − supervised lexical	+22.8	[17.3, 28.2]	[5.8, 40.2]
Factor deterministic − embedding	−11.6	[−15.1, −8.0]	[−26.2, 2.3]

Table S32. Paired changes in commitment formation and declared fulfillment, in percentage points. Both use the same source resampling as the task-success comparison.

Contrast	Metric	Difference	Stratified 95% CI	Unstratified 95% CI
Paraphrases				
Factor: deterministic − learned	C	0.0	[0.0, 0.0]	[0.0, 0.0]
Factor: deterministic − learned	F	0.0	[−0.2, 0.2]	[−0.3, 0.3]
Direct: deterministic − learned	C	0.0	[0.0, 0.0]	[0.0, 0.0]
Direct: deterministic − learned	F	0.0	[0.0, 0.0]	[0.0, 0.0]
Factor deterministic − Direct learned	C	+0.2	[−1.9, 2.3]	[−8.4, 6.9]
Factor deterministic − Direct learned	F	+0.1	[−2.0, 2.2]	[−8.5, 6.9]
Factor deterministic − TF–IDF	C	−6.2	[−6.2, −6.2]	[−15.6, 0.0]
Factor deterministic − TF–IDF	F	−6.0	[−6.2, −5.8]	[−15.6, 0.4]
Factor deterministic − supervised lexical	C	−6.2	[−6.2, −6.2]	[−15.6, 0.0]
Factor deterministic − supervised lexical	F	−6.4	[−6.5, −6.2]	[−15.7, 0.0]
Factor deterministic − embedding	C	−6.2	[−6.2, −6.2]	[−15.6, 0.0]
Factor deterministic − embedding	F	−6.4	[−6.5, −6.2]	[−15.7, 0.0]
With distractors				
Factor: deterministic − learned	C	0.0	[0.0, 0.0]	[0.0, 0.0]
Factor: deterministic − learned	F	−0.3	[−0.6, 0.0]	[−0.7, 0.1]
Direct: deterministic − learned	C	0.0	[0.0, 0.0]	[0.0, 0.0]
Direct: deterministic − learned	F	−0.3	[−0.6, 0.0]	[−0.7, 0.1]
Factor deterministic − Direct learned	C	+1.3	[−1.0, 3.5]	[−7.9, 8.3]
Factor deterministic − Direct learned	F	+0.9	[−1.4, 3.2]	[−8.1, 8.1]
Factor deterministic − TF–IDF	C	−6.2	[−6.2, −6.2]	[−15.6, 0.0]
Factor deterministic − TF–IDF	F	−6.0	[−6.8, −5.3]	[−15.7, 0.6]
Factor deterministic − supervised lexical	C	−6.2	[−6.2, −6.2]	[−15.6, 0.0]
Factor deterministic − supervised lexical	F	−6.0	[−6.7, −5.4]	[−15.7, 0.6]
Factor deterministic − embedding	C	−6.2	[−6.2, −6.2]	[−15.6, 0.0]
Factor deterministic − embedding	F	−6.7	[−7.0, −6.4]	[−16.0, −0.3]

formation, actions, and readback in episode time, whereas TTC ends at commitment formation or rejection. Table S33 and Figure S3(b) use outcomes from this same timed sample.

Table S33. Latency and verified deadline success on the fresh sample. Times are seconds over all assigned trials, including rejection time. TTC ends at commitment, episode time includes world execution, and $V(30)/V(60)$ give percentages completing the task by 30/60 s.

Configuration	n	G	TTC mean	TTC median	TTC p95	Episode mean	Episode median	$V(30)$	$V(60)$
Direct / learned	192	125	2.35	2.29	2.56	24.09	24.98	64.6	65.1
Direct / deterministic	192	125	1.57	1.55	1.72	23.29	24.18	65.1	65.1
Factor / learned	192	154	0.78	0.78	0.90	22.31	23.55	80.2	80.2
Factor / deterministic	192	154	0.44	0.43	0.51	21.97	23.14	80.2	80.2
TF–IDF retrieval	64	42	0.07	0.06	0.11	22.94	22.66	65.6	65.6
Supervised lexical	64	42	0.34	0.06	0.13	23.21	22.68	64.1	65.6
Embedding retrieval	64	58	0.12	0.10	0.18	22.99	22.69	90.6	90.6

Table S34 reports the change caused by replacing learned reception with deterministic realization, which removes one generation step. We average roles and checkpoints within source before 10,000 paired bootstrap resamples, reporting TTC and complete episode time as distinct outcomes.

The lexical classifier's latency distribution includes a 17.33 s TTC observation despite a 5.35 ms selection service measurement, and that value remains in all summaries. Factor with deterministic reception has a TTC advantage over learned Direct, while their episode-time interval spans zero because episode duration also includes the environmental work.

Table S34. Paired changes in TTC and episode time, in milliseconds. A negative value denotes shorter time for the first configuration, with every assigned repeat included.

Contrast	Δ TTC (ms)	95% CI	Δ episode (ms)	95% CI
Factor: deterministic − learned	−343	[−374, −292]	−335	[−378, −279]
Direct: deterministic − learned	−785	[−936, −681]	−800	[−956, −680]
Factor deterministic − Direct learned	−1, 918	[−2070, −1812]	−2, 114	[−5679, 540]
Factor deterministic − TF–IDF	+371	[359, 383]	−963	[−3891, 634]
Factor deterministic − supervised lexical	+99	[−448, 379]	−1, 239	[−4291, 543]
Factor deterministic − embedding	+319	[301, 335]	−1, 016	[−3926, 529]

Table S35 counts work per timed trial, distinguishing model generation from query embedding. The retrieval index encodes 64 documents during initialization, while each online request performs one query encoding. The reported peer traffic comprises the Short DSL request and its acceptance acknowledgment, making it directly interpretable as communication between agents.

Table S35. Mean work per assigned timed trial. Generation tokens and embedding-query tokens are distinguished, and peer traffic comprises the transmitted request and acknowledgment.

Configuration	Gen. calls	Encoder calls	Input tokens	Output tokens	Query tokens	REQ bytes	ACK bytes
Direct / learned	1.95	0.00	943.1	76.9	0.0	71.5	64.5
Direct / deterministic	1.00	0.00	455.0	53.3	0.0	71.5	64.5
Factor / learned	1.94	0.00	968.1	18.5	0.0	72.4	63.8
Factor / deterministic	1.00	0.00	457.0	9.6	0.0	72.4	63.8
TF–IDF retrieval	0.00	0.00	0.0	0.0	0.0	75.2	68.0
Supervised lexical classifier	0.00	0.00	0.0	0.0	0.0	75.2	68.0
Embedding retrieval	0.00	1.00	0.0	0.0	37.4	76.4	68.0

S9.5 Verified Success by Deadline and Semantic Choice

We define deadline success as $V(t) = N^{-1} \sum_i \mathbf{1}\{G_i = 1, T_{\text{episode},i} \le t\}$, the fraction of assigned trials that complete the original task by time t. Unsubmitted or unsuccessful trials contribute zero, so Table S33 measures completion by a deadline rather than latency conditional on success.

Figure S3(c) shows that errors concentrate differently across semantic choices. Factor fulfills the wrong storage-branch assignment in all 120 trials for that choice, whereas the crafting-table choice fails to form a complete sender object in all 120. Embedding retrieval instead reaches 30.0% success for Site B and 50.0% for Chest B. These distributions distinguish selection

errors from incomplete generation and suggest that selector behavior depends on how a task choice is expressed. Each choice has two source phrasings, with complete source and checkpoint outcomes in the accompanying data.

We then use the new descriptions in Section S11 to test the same checkpoints and retriever under another language distribution, invoking additional generation according to a fixed score threshold.

S10 Descriptions at the Planning Layer

This section supports assignment from language in main Section 6.3 by evaluating the prediction step before receiver realization. We provide complete method results and source-level uncertainty, then connect those predictions to the online experiment in Section S9.

S10.1 Binding from Descriptions

We replace the explicit assignment field with a description of the private fact, keeping two possible assignments per template and all 16 semantic choices represented in training. Syntactic recombination evaluates new expressions through Minecraft execution, while paraphrases change lexical wording and distractors add irrelevant material to the same source phrasings. Table S36 distinguishes those outcomes, and Section S9 follows the paraphrase conditions through the complete online protocol.

Table S36. Assignment from descriptions under three language conditions. Syntactic recombination reports Minecraft task success, while paraphrase and distractor columns report planning accuracy. Each uses 320 role instances and three checkpoints for learned methods, with the latter two conditions derived from 32 phrasings across 16 choices.

Formation method	Recombination: task success	Paraphrases: binding accuracy	With distractors: binding accuracy
Direct Short DSL (0.8B)	100.0%	65.8%	64.8%
Learned factor + rule (0.8B)	100.0%	**79.1%**	**80.1%**
Exact-code parser rule	100.0%	0.0%	0.0%
TF–IDF nearest-neighbour rule	100.0%	59.4%	62.5%
Supervised TF–IDF + logistic regression	—	59.4%	57.5%
Complete surface table	0.0%	0.0%	0.0%
Oracle factors + rule	100.0%	100.0%	100.0%

The exact-code parser reads an explicit code, whereas the surface table looks up a complete known string. Table S36 compares these mechanisms with learned and retrieved assignment selection, with paired source uncertainty in Section S10.2. This separation identifies what changes when the input communicates the fact through wording rather than a supplied code.

S10.2 Source Inference and Supervised Classification

S10.2.1 Estimand and Source Hierarchy. Each language condition contains 160 world clusters and 320 role instances derived from 32 original phrasings, with two phrasings for each of 16 template–assignment choices. The distractor condition modifies the same sources. We average worlds, roles, and checkpoints within phrasing before computing paired differences, so repeated executions do not become additional language sources.

Our primary estimand gives each semantic choice equal weight, averaging its two source differences before combining choices. For paired difference $\bar{d}_{cs}$, we therefore use

$$\widehat{\Delta} = \frac{1}{16}\sum_{c=1}^{16}\frac{1}{2}\sum_{s=1}^{2}\bar{d}_{cs}.$$

We obtain primary intervals by resampling phrasings within choice, while unstratified intervals resample all 32 phrasings and allow the semantic mixture to change. Table S37 reports both schemes using 10,000 percentile resamples conditional on the evaluated checkpoints, because the schemes represent different sources of uncertainty.

Table S37. Paired planning-accuracy differences in percentage points. Source-stratified intervals hold the mixture of 16 choices fixed, while unstratified intervals allow that mixture to vary. Retrieval denotes TF–IDF nearest-neighbor lookup.

Contrast	Difference	Stratified 95% CI	Unstratified 95% CI
Paraphrases: Factor − retrieval	19.7	[12.0, 27.3]	[2.3, 37.8]
With distractors: Factor − retrieval	17.6	[11.0, 24.0]	[0.4, 35.2]
Paraphrases: Direct − retrieval	6.5	[−0.4, 13.3]	[−7.5, 21.8]
With distractors: Direct − retrieval	2.3	[−3.5, 8.1]	[−12.1, 17.0]

A source-level standard-error sensitivity gives intervals of $[0.8, 38.6]$ points for Factor minus retrieval on paraphrases and $[-1.2, 36.4]$ with distractors, using approximately 2.04 standard errors. The distractor interval and Direct's stratified intervals include zero, illustrating the uncertainty that remains with two source phrasings per semantic choice.

S10.2.2 Supervised Lexical Baseline. We build the supervised lexical classifier from word 1–2-grams and character-within-word 3–5-grams using TF–IDF and template-conditioned L2 logistic regression. We fit the vocabulary and inverse-document frequencies on 64 training descriptions and select regularization from $\{0.1, 1, 10\}$ on 16 development descriptions. Development accuracy selects 0.1, with the smaller value used to resolve ties.

The classifier correctly predicts 190/320 role instances on paraphrases and 184/320 with distractors, as shown in Table S36. The first condition contains 32 surface strings, of which 19 are correct, and the second contains 64 strings derived from the same 32 sources. The online experiment applies the same fitted classifier before physical execution.

S11 Selection before Commitment

This section supports the assignment choices in main Section 6.3 by testing fixed selectors on new descriptions and measuring selective use of additional generation. We report the complete prospective comparison with Factor, followed by assignment accuracy and decision costs when DeepSeek supplies the hosted alternative.

S11.1 Descriptions and Prospective Design

We generate descriptions with a language model from task specifications and screen them for a unique intended assignment. The corpus contains 928 descriptions: 32 for execution checks, 256 for calibration, and 640 for testing. Each of 16 choices has 16 calibration descriptions and 40 test descriptions, with the split fixed before scoring.

We keep the same three Factor checkpoints, Qwen3-Embedding-0.6B retriever, and 64-description training index, while expressing the tasks through newly written descriptions. For example, a description may require a crafting table while explicitly mentioning that a chest would be unsuitable. This changes the wording of a familiar choice without retraining either selector.

We calibrate on the difference between retrieval scores for the two legal assignments, selecting the 52nd smallest margin among 256 calibration descriptions. The resulting threshold, approximately 0.00942109, uses strict inequality, so the tied threshold value stays with retrieval. It routes 97/640 test descriptions to additional generation, or 15.2%, with the threshold fixed before their scores are evaluated.

S11.2 Strategies and Physical Outcomes

Retrieval selects an assignment and composes its request, whereas Factor predicts the assignment code before the same deterministic composition. Selective upgrading first runs retrieval and invokes Factor when the score margin falls below the threshold. Factor receives the original local description, and its output is checked before transmission. The implementation uses the retrieval choice if that output is inadmissible, although every Factor output in this test passes the check.

The random and maximum-similarity controls upgrade the same number of descriptions within each public template, allowing us to compare how each ranking spends a fixed quota. All roles and checkpoints share the description's routing decision. The abstention strategy instead submits above the same margin threshold, with unsubmitted trials counted as unsuccessful assignments.

Table S38 reports all six strategies across 17,920 assigned trials, including two roles per description and three checkpoints for strategies that use Factor. The deterministic receiver checks the transmitted request using projected handoff context, after which Minecraft supplies actual transfer and terminal readback. All 17,726 formed commitments are physically fulfilled, while 2,103 select work inconsistent with the original task and the remaining 194 assignments abstain before formation.

Table S38. Prospective task outcomes for all six strategies. C, H, and F coincide within each row, while $F - G$ counts fulfilled commitments that select the wrong work. Factor use is the percentage of assigned trials that invoke its generator.

Strategy	Assigned	$C = H = F$	G	G (%)	$F - G$	Factor (%)
Embedding retrieval	1,280	1,280	1,052	82.2	228	0.0
Factor	3,840	3,840	3,684	95.9	156	100.0
Margin-guided upgrade	3,840	3,840	3,395	88.4	445	15.2
Random upgrade	3,840	3,840	3,268	85.1	572	15.2
Score-guided upgrade	3,840	3,840	3,280	85.4	560	15.2
Margin-based abstention	1,280	1,086	944	73.8	142	0.0

S11.3 Ranking and Paired Effects

We average roles and checkpoints within each test description, then resample the 640 descriptions within their 16 semantic choices for 10,000 paired draws. The two primary effects use 97.5% intervals, while secondary contrasts use 95%. Unstratified resampling provides a sensitivity analysis over the same independent description unit.

We measure ranking by pairing a correctly and an incorrectly selected description from the same public template, giving tied confidence scores half weight. The resulting AUROC is 0.771 for the margin and 0.595 for maximum similarity. Pooling across templates instead gives 0.757 and 0.570, with the within-template comparison used for primary inference.

Table S39 compares selective upgrading with retrieval, both matched-quota controls, and full Factor. Margin ranking improves how the quota is spent, although full Factor provides the highest task success. The paired quality gap and the decision-time differences locate selective upgrading between retrieval and full Factor, improving retrieval while using fewer generation calls.

Table S39. Paired prospective effects over 640 descriptions. Confidence levels are shown per row, and both interval schemes average repeated roles and checkpoints within description.

Contrast	Metric	Effect	Level (%)	Stratified CI	Unstratified CI
Margin − maximum similarity	AUROC	+0.176	97.5	[0.109, 0.245]	[0.107, 0.247]
Upgrade − retrieval	G (pp)	+6.2	97.5	[4.4, 8.2]	[4.2, 8.4]
Upgrade − random	G (pp)	+3.3	95	[1.5, 5.2]	[1.5, 5.2]
Upgrade − score	G (pp)	+3.0	95	[1.1, 4.9]	[1.1, 4.9]
Upgrade − Factor	G (pp)	-7.5	95	[−9.9, −5.2]	[−10.0, −5.1]

Margin-based abstention submits 1,086 of 1,280 assigned requests and succeeds in 944, giving 84.8% coverage and 73.8% all-assigned task success. The submitted error rate is 13.1%, corresponding to 71 errors among 543 distinct submitted description choices. These values characterize the accepted workload under the fixed quantile threshold.

S11.4 Same-Card Timing and Calls

We use 64 descriptions for timing, four per choice, and cross their two roles with the existing checkpoints where applicable. Resident retrieval and Factor models run on the same RTX 4090D with batch size one and a Latin-square method order. Timing begins after warm-up and world setup, measuring formation to commitment and then the additional environmental work through terminal readback.

Table S40 reports 1,024 timed trials, with 1,008 formed commitments and 1,007 fulfilled commitments because one Factor trial fails at declared completion. Selective upgrading invokes Factor on 12.5% of this timed subset, compared with 15.2% in the quality sample. Table S41 pairs the time, quality, and call differences after averaging roles and checkpoints within description.

Table S40. Formation time, physical outcomes, and generator use on the fresh timing sample. TTC is mean milliseconds, episode time is mean seconds, and Factor calls are per assigned trial.

Strategy	n	C	H	F	G	TTC	Episode	Factor calls
Embedding retrieval	128	128	128	128	116	91	22.98	0.000
Margin-guided upgrade	384	384	384	384	366	144	23.01	0.125
Factor	384	384	384	383	373	448	23.30	1.000
Margin-based abstention	128	112	112	112	106	93	20.20	0.000

Table S41. Paired effects on the fresh sample with 95% intervals over 64 descriptions. Differences are first strategy minus second, so negative timing effects denote savings.

Contrast	Metric	Difference	95% CI
Margin upgrade minus retrieval	Success (pp)	+4.7	[0.0, 9.4]
Margin upgrade minus retrieval	TTC (ms)	+53	[34, 71]
Margin upgrade minus retrieval	Episode (ms)	+29	[−9, 64]
Margin upgrade minus retrieval	Factor calls	+0.125	[0.078, 0.172]
Margin upgrade minus Factor	Success (pp)	-1.8	[−6.5, 2.3]
Margin upgrade minus Factor	TTC (ms)	-304	[−355, −261]
Margin upgrade minus Factor	Episode (ms)	-286	[−350, −227]
Margin upgrade minus Factor	Factor calls	-0.875	[−0.922, −0.828]
Abstention minus retrieval	Success (pp)	-7.8	[−12.5, −3.1]
Abstention minus retrieval	TTC (ms)	+1	[−3, 5]
Abstention minus retrieval	Episode (ms)	-2789	[−3923, −1675]
Abstention minus retrieval	Factor calls	+0.000	[0.000, 0.000]

S11.5 Semantic Choices and Checkpoints

Table S42 reports each choice's outcomes over 40 descriptions, corresponding to 80 role trials for retrieval and 240 role–checkpoint trials for Factor strategies. Table S43 gives the checkpoint decomposition, allowing readers to see which assignments benefit from additional generation and how that pattern varies across trained models.

Table S42. Task success by semantic choice, in percent. The three upgrade strategies have matched quotas, and abstention counts unsubmitted trials as unsuccessful.

Choice	Retrieval	Factor	Margin	Random	Score	Abstain
Active order: storage	100.0	98.3	100.0	100.0	100.0	100.0
Active order: work	65.0	99.2	75.0	65.0	70.0	60.0
Build site A	92.5	97.5	95.0	95.0	95.0	82.5
Build site B	82.5	100.0	95.0	82.5	90.0	40.0
Chest A	100.0	100.0	100.0	100.0	100.0	85.0
Chest B	40.0	100.0	87.5	75.0	72.5	10.0
Recipe: chest	87.5	94.6	90.8	87.5	90.0	87.5
Recipe: crafting table	97.5	100.0	97.5	97.5	97.5	97.5
Allocation: build	87.5	94.2	87.5	87.5	87.5	87.5
Allocation: deposit	92.5	100.0	92.5	92.5	92.5	92.5
Build component: roof	97.5	94.6	97.5	97.5	97.5	92.5
Build component: wall	77.5	96.7	80.0	77.5	75.0	77.5
Recipe: planks	97.5	94.2	100.0	95.8	95.0	97.5
Recipe: sticks	50.0	100.0	65.0	60.0	55.0	37.5
Terminal: depot	60.0	66.7	58.8	60.8	59.2	52.5
Terminal: marker	87.5	99.2	92.5	87.5	90.0	80.0

Table S43. Task success for the existing Factor checkpoints, with counts out of 1,280 followed by percentages. The retrieval reference has 1,052 successes for every checkpoint comparison.

Checkpoint	Factor	Margin	Random	Score
Checkpoint 1	1,216 (95.0)	1,133 (88.5)	1,088 (85.0)	1,094 (85.5)
Checkpoint 2	1,207 (94.3)	1,126 (88.0)	1,086 (84.8)	1,090 (85.2)
Checkpoint 3	1,261 (98.5)	1,136 (88.8)	1,094 (85.5)	1,096 (85.6)

S11.6 Routing Outcomes

Table S44 distinguishes retrieval errors corrected by upgrading from correct retrieval choices changed into errors. Both the upgraded and retained groups contain errors that Factor can correct, explaining why selective use improves retrieval while full Factor reaches higher quality. The accompanying data also provide the writing-task sensitivity analysis.

Table S44. Assignment changes caused by replacing retrieval with Factor. Corrected and introduced errors use matched descriptions and roles, with full Factor providing the comparison on descriptions retained by retrieval.

Checkpoint	Routing group	Trials	Corrected	Changed to wrong	Both wrong	Both correct
1	Retrieval kept	1086	115	32	27	912
1	Upgraded	194	82	1	4	107
2	Retrieval kept	1086	117	36	25	908
2	Upgraded	194	80	6	6	102
3	Retrieval kept	1086	138	13	4	931
3	Upgraded	194	84	0	2	108

S11.7 Local and Hosted Assignment Models

We also apply a prompted DeepSeek selector to the same 640 test descriptions, comparing it with embedding retrieval, the unchanged Factor checkpoints, and retrieval with low-margin escalation to DeepSeek. The common composer expresses each selector's chosen assignment through the same executable request interface. Table S45 evaluates the chosen binding against the intended binding before world execution, which directly measures the decision made by the formation backend.

DeepSeek selects correctly in 3,839 of 3,840 trials, compared with 3,684 for Factor and 1,052 of 1,280 for retrieval. Routing the 97 lowest-margin descriptions to DeepSeek selects correctly in 3,414 of 3,840 trials. The random-upgrade value is an expectation

Table S45. Assignment accuracy and measured decision cost. Correct assignments are determined before execution on 640 descriptions, with two roles and three repetitions for generative selectors. Timing uses 128 decisions per strategy. API charges are usage estimates.

Selector	Correct/assigned	Accuracy (%)	Time (ms)	API USD
Embedding retrieval	1,052/1,280	82.19	92	0.00000
Factor	3,684/3,840	95.94	412	0.00000
DeepSeek	3,839/3,840	99.97	1402	0.00009
Retrieval + DeepSeek	3,414/3,840	88.91	350	0.00001
Random upgrade (expectation)	–	84.88	–	–

under the same overall upgrade probability, computed from each description's retrieval and DeepSeek outcomes. Consequently it denotes a policy expectation over the observed decisions, rather than an additional set of executed trials.

For the decision-time comparison, all four strategies have 128 measured observations. Table S45 reports the mean elapsed decision time and estimated API charge, including query encoding, any hosted generation, and the common formation checks within the recorded timing interval. Local methods incur zero API charges, while their local computation is represented by the measured latency. The timing subset invokes DeepSeek on 17 of 128 routed decisions, whereas the quality set routes 97 of 640 descriptions, so each sample's call rate follows its own inputs.

Although full DeepSeek attains the highest assignment accuracy, Factor uses about 29% of its measured mean decision time and performs the decision locally. Selective escalation improves retrieval at an intermediate operating point, while Factor remains more accurate than that routed policy. Together with the closed-loop task results on template-derived descriptions in Section S9, these comparisons show why the common interface is useful: the selector can be chosen for the language distribution and deployment objective without changing what constitutes an executable commitment.

S12 Protocol Diagnostics

This section supports the protocol checks associated with main Sections 3 and 5.3 by testing which assignment a delivered request or response causes the system to execute. The full-information reference and information-removal conditions provide complementary checks of the local contexts used in those interactions.

Figure S4 summarizes the local request and feedback diagnostics. We preserve the original task requirements when substituting a legal message, so the plots show how communication selects the tested assignment while execution and fulfillment are checked through the protocol.

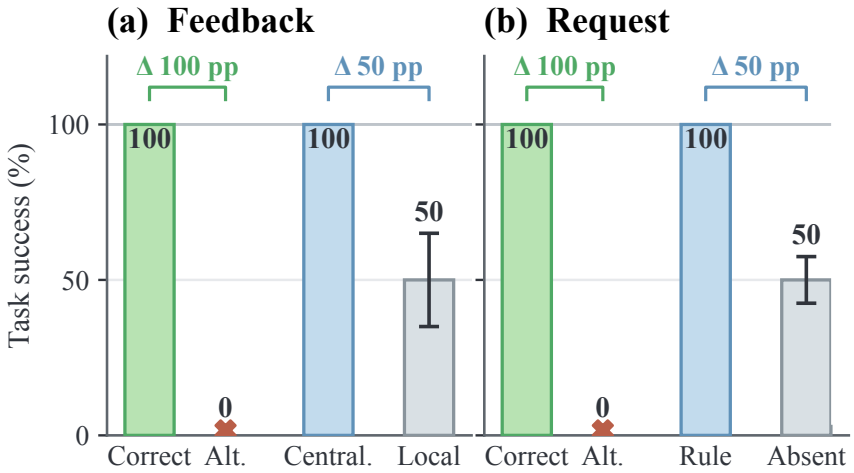


Figure S4. Local content-substitution checks. (a) Correct and alternative feedback select different workcell assignments, while local planning and centralized information provide information references. (b) Correct, absent, and alternative requests test whether the receiver selects the branch communicated by the sender. Each axis measures success against the original task.

S12.1 Paired Local Information

The local-information bound supports the diagnostic construction described in main Section 3. Consider equiprobable tasks z_1, z_2 with the same local view and disjoint acceptable route sets $\mathcal{P}_1, \mathcal{P}_2$. A policy choosing a route from that view has a common route distribution μ, and therefore

$$\frac{\mu(\mathcal{P}_1) + \mu(\mathcal{P}_2)}{2} \leq \frac{1}{2}.$$

The inequality follows because probabilities assigned to disjoint sets sum to at most one. We use the bound to construct a balanced check of whether a delivered request or response supplies the information needed for the prescribed route.

To quantify message substitution, let $Y_z(m)$ denote task success after message m is delivered in task z, and let $\overline{Y}_z$ average its repeated role and model evaluations. We replace the true message m_z with a legal alternative $\widetilde{m}_z$ under the same world and call procedure, defining

$$\widehat{\Delta}_{\text{content}} = \frac{100}{|\mathcal{Z}|} \sum_{z \in \mathcal{Z}} \left[\overline{Y}_z(m_z) - \overline{Y}_z(\widetilde{m}_z) \right]. \tag{S4}$$

Both outcomes use the original task specification, so the difference measures the consequence of the delivered content while fulfillment is evaluated against the commitment actually formed.

S12.2 Checking Failure Stages

We validate the stage predicates by applying controlled changes to a proposal, a handoff, or the work after receipt, with 256 assigned trials for each of nine conditions. Table S46 reports the common outcome within each condition, distinguishing a rejected proposal from an unsuccessful physical transfer and a completed but inappropriate assignment. The clean and irrelevant-change controls each have zero alarms in 256 trials.

Table S46. Stage checks under controlled faults. Every row describes 256 trials with the shown outcome, where a dash denotes a stage that is not reached. T is declared terminal completion.

Condition	Assigned	C	H	T	F	G
Clean	256	1	1	1	1	1
Irrelevant change	256	1	1	1	1	1
Malformed proposal	256	0	–	–	–	0
Incompatible handoff	256	0	–	–	–	0
Insufficient handoff	256	1	0	–	0	0
Wrong transfer item	256	1	0	–	0	0
Resource loss after receipt	256	1	1	0	0	0
Terminal action failure	256	1	1	0	0	0
Compatible wrong assignment	256	1	1	1	1	0

Every injected condition is detected at the corresponding check, and independently evaluated predicates agree with those used by the execution pipeline. This verifies the measurement tools used to locate failure stages in the communication comparisons, including the distinction between fulfilling an accepted request and satisfying the original task.

S12.3 Request Content

The true and alternative requests in Table S47 preserve message presence, schedule, schema, approximate length, checkpoint, and calls, so the contrast changes delivered content. The absent-request reference instead uses an independently trained model that plans its own work, while the zero-call rule tests execution from an explicitly supplied correct assignment.

Table S47. Request substitutions and a deterministic assignment reference over 160 clusters. Learned methods use three checkpoints and 960 episodes, while the rule uses 320 role views. Differences are true request minus each condition, in percentage points. Bytes count canonical request objects rather than Short DSL surface strings.

Condition	Episodes	Success	Calls	Request (B)	Δ vs. true	95% CI
True learned request	960	100.0%	2.0	304.4	–	–
Request removed (self-plan-only)	960	50.0%	2.0	0.0	+50.0	[42.5, 57.5]
Same-template executable alternative	960	0.0%	2.0	304.4	+100.0	[100, 100]
Deterministic correct binding	320	100.0%	0.0	304.4	0.0	–

Replacing the request with the other legal option makes the receiver follow that option in 960/960 diagnostic episodes. The held-out pool similarly preserves the delivered assignment in all 471 executable outputs, while nine trials stop before forming an executable plan. These observations check consistency between the message, admission, and the branch subsequently realized.

S12.4 Feedback and Full Information

S12.4.1 Paired Construction and Matched Training. We cross two goals, two prescribed processing modes, and ten worlds to obtain 40 clusters, with 20 goal–world pairs sharing the requester's input and initial proposal. Two roles and three checkpoints give 240 trials per condition, or 960 across the four conditions. Primary intervals average repetitions within each cluster, and sensitivity intervals also average the paired modes. Tables S48, S49, and S50 connect the training configuration, task outcomes, and affected fields, while Figure S5(c) displays individual clusters.

Table S48. Workcell feedback and centralized training budgets. Token occurrences include prompt, target, and end markers across the training epochs.

Model	Visible information	Rows	Epochs	Updates	Train-token occurrences	Seeds
Distributed requester	320 initial local + 320 after-response local	640	2	80	868,720	3
Centralized full information	merged current local views	320	4	80	970,080	3

Distributed supervision contains 320 initial proposals and 320 revisions, whereas the centralized reference receives 320 joint-plan targets. Both use Qwen3.5-0.8B, effective batch size 16, learning rate 10^{-5}, and maximum length 2,048 with final-step checkpoints. At matched updates the centralized model processes 1.117 times as many token occurrences, including prompts, targets, and end markers, and every example fits the length limit.

Table S49. Workcell task success and information components over 40 clusters forming 20 mode pairs. Intervals use the cluster bootstrap, and strict success requires all six role–checkpoint repetitions to succeed. The final column measures the stated message component.

Condition	Seeds	Epis.	Success	95% CI	Strict	Calls	Measured boundary component
Requester-local (A-only)	3	240	50.0%	[35, 65]	20/40	1.0	0 B cross-boundary payload
Correct bounded feedback	3	240	100.0%	[100, 100]	40/40	2.0	response: mean 103 B
Counterfactual feedback	3	240	0.0%	[0, 0]	0/40	2.0	injected response: mean 103 B
Centralized full information	3	240	100.0%	[100, 100]	40/40	1.0	state aggregation: mean 956 B

S12.4.2 Feedback-Content Intervention. Correct feedback resolves the assignment left ambiguous by local planning, while an admissible counterfactual redirects the work under the same two-call procedure (Table S49). Pair-block sensitivity gives a 50.0-point gain [50, 50] over local planning and a 100.0-point gain [100, 100] over the counterfactual. Every checkpoint reproduces the 50/100/0 profile. The 720 feedback trials parse and complete planning, and all 240 counterfactual trials complete transfer and deposit while violating the prescribed actor and handoff form.

Table S50. Assignment fields after counterfactual feedback, separating the altered division of work from the unchanged requested product.

Field	Matches injected	Matches true world	Observed behavior
Craft actor	240/240	0/240	rewritten
Handoff item	240/240	0/240	rewritten
Handoff count	240/240	0/240	rewritten
Peer suffix	240/240	0/240	rewritten
Goal item	240/240	240/240	invariant
Goal count	240/240	240/240	invariant

Because the intervention fixes the requester's context, initial proposal, world, decoding, and call schedule, Table S50 identifies the allocation fields controlled by the response. The craft actor, handoff object and count, and continuation follow the injected alternative in all 240 counterfactual trials, while the goal object and count stay fixed.

The data labels `RAW_PROCESSOR` and `FINISHED_RECEIVER` correspond to Mode B-crafts and Mode A-crafts, with the first mode assigning the peer to process transferred material. In the second, `CRAFT_AT_REQUESTER_HANDOFF_FINISHED` names the alternative that moves crafting before transfer, allowing both routes to produce the required item under different assigned responsibilities.

S12.4.3 Centralized Full-Information Reference. We supply both local views to one model call in the centralized reference, which succeeds in 240/240 trials and matches the correct-feedback outcome. Its mean state upload is 955.5 B, with a 963 B 95th percentile, whereas the response component averages 103 B with a 118 B 95th percentile. Since these components carry different information, the complete accounting below sums uploads, downloads, and peer exchanges for each protocol.

For architecture accounting, let $\mathcal{A}_t \subseteq \mathcal{A}$ be the participating agents at round t, let $E_t \subseteq \mathcal{A} \times \mathcal{A}$ be the directed active-edge set, and let $\mathcal{M}_{ij}^t$ be the complete sequence of application messages sent on directed edge (i, j). The function $b(m)$ counts canonical UTF-8 payload bytes, including any application wrapper present in m but excluding transport framing. Let $m_{i,\uparrow}^t$ and $m_{i,\downarrow}^t$ denote centralized upload and download payloads. With R_c centralized rounds and R_l local rounds,

$$B_{\text{central}} = \sum_{t=1}^{R_c} \sum_{i \in \mathcal{A}_t} \left[b(m_{i,\uparrow}^t) + b(m_{i,\downarrow}^t) \right], \tag{S5}$$

$$B_{\text{local}} = \sum_{t=1}^{R_l} \sum_{(i,j) \in E_t} \sum_{m \in \mathcal{M}_{ij}^t} b(m). \tag{S6}$$

Feedback sent in the reverse direction belongs to the corresponding edge (j, i). Totals depend on coordination rounds; peer traffic also depends on active edges and messages per edge.

S12.5 Request Transfer to a Prompted External Model

This section supports the use of an external formation backend by giving the complete request-transfer outcomes and the prompts used in that diagnostic. We preserve the executable interface while changing how the receiver produces its continuation, so the complete outcomes can be interpreted against the same task requirements.

S12.5.1 Instances, Conditions, and Statistical Units. We construct the request-transfer set from eight templates and ten paired world blocks per template, each with two assignments and two role views. Three evaluations per role view produce 960 trials for each condition and 7,680 across both backends and four conditions. API repetitions call the same service, whereas local repetitions use the three unchanged Direct checkpoints. Test worlds are distinct from the request pool in Section S12.3 and from development worlds.

Correct, absent, and alternative request trials share an upstream plan and the receiver's context before delivery, varying the message supplied at that point. Here removal uses the same backend, unlike the separately trained absent-request reference in Section S12.3. In the complete protocol, sender and receiver generate sequentially from their own inputs. A legal path matching the injected alternative can proceed to execution, whereas a path contradicting the delivered request is rejected before commitment formation.

For inference we average the two assignments, two roles, and three repetitions within each world block, then resample ten blocks within each template for 10,000 draws. The true-minus-alternative contrast measures replacement under the same procedure, whereas true-minus-absent also changes message presence. Both contrasts keep all assigned trials in the task-success calculation.

Table S51. Prompted and local request-transfer outcomes over 960 trials per condition. Formation and success are counts with percentages, while failure columns partition sender formation, receiver formation, request conflict, and formed commitments failing the original task.

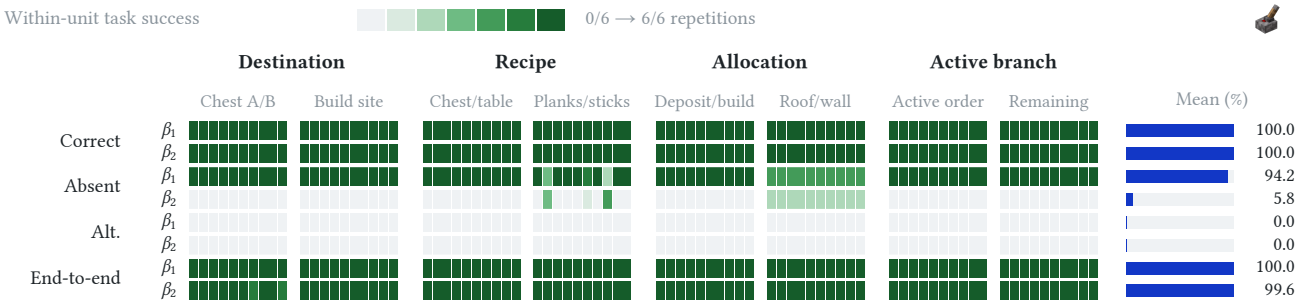
Backend	Condition	Formed (%)	Success (%)	S	R	C	W
DeepSeek	Correct request	949 (98.9)	949 (98.9)	0	9	2	0
DeepSeek	Absent request	795 (82.8)	385 (40.1)	0	165	0	410
DeepSeek	Alternative request	957 (99.7)	0 (0.0)	0	3	0	957
DeepSeek	Full protocol	952 (99.2)	952 (99.2)	1	7	0	0
Local	Correct request	960 (100.0)	960 (100.0)	0	0	0	0
Local	Absent request	960 (100.0)	480 (50.0)	0	0	0	480
Local	Alternative request	960 (100.0)	0 (0.0)	0	0	0	960
Local	Full protocol	958 (99.8)	958 (99.8)	2	0	0	0

Table S52. Paired task-success differences in percentage points, with 95% intervals from resampling 80 paired world blocks within template.

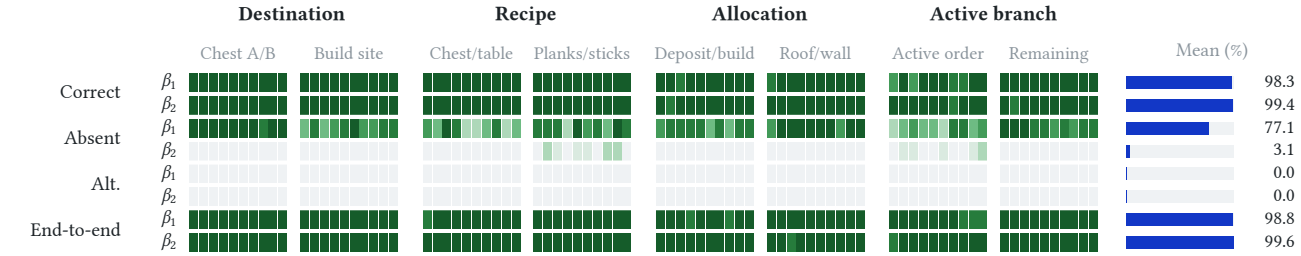
Backend	Contrast	Effect (pp)	95% CI
DeepSeek	Correct minus legal alternative	+98.9	[98.1, 99.5]
DeepSeek	Correct minus absent	+58.8	[56.8, 60.8]
Local	Correct minus legal alternative	+100.0	[100.0, 100.0]
Local	Correct minus absent	+50.0	[50.0, 50.0]

S12.5.2 Content Effects and Failure Location. Tables S51 and S52 show how request use carries from a fine-tuned local model to prompted external generation. The prompted correct-request failures comprise nine invalid receiver formations, including seven in Active Branch, and two legal branches that contradict the request. The examples below distinguish those contradictions from syntax errors and include them as unsuccessful trials.

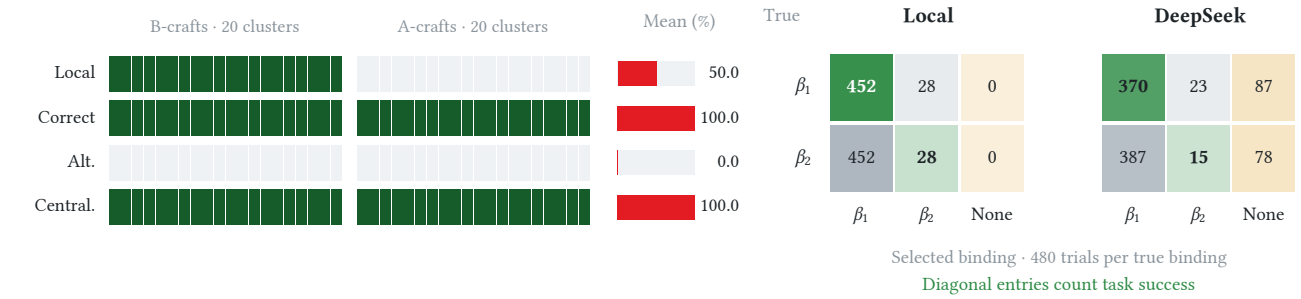
Figure S5 expands these aggregate outcomes to individual world blocks, relating the absent-request behavior to the selected assignment and comparing it with the feedback clusters.



(a) Request · local checkpoints · 80 world blocks × 2 bindings



(b) Request · few-shot DeepSeek API · the same 80 world blocks



(c) Feedback · 40 semantic clusters

(d) Absent request · route selection

Figure S5. Request and feedback outcomes by evaluation unit. Panels (a,b) show the local and prompted backends on 80 paired world blocks, and panel (c) shows 40 workcell clusters by processing mode. Each cell averages six role–repetition or role–checkpoint trials, with pooled rates beside the rows. Panel (d) relates the choice made without a request to the true assignment, while None denotes the absence of a formed plan. The outcomes provide a unit-level check of the paired-information construction in Section S12.1.

Two legal branches that violate the request. In the Deposit-or-Build example, the receiver is asked to construct an object but generates a deposit instead. The displayed request and output are the transmitted and generated strings from the first API repetition, with Agent B sending to Agent A:

```
Delivered:
REQ agent_a build.component(bind=BUILD,q=1,item=cobblestone,site=marker_site)
Generated:
SELF resource.deliver(bind=DEPOSIT,q=4,dst=reserve_chest,item=cobblestone)
REQ -
```

The Remaining Terminal example has the same sender and receiver roles, but substitutes a depot continuation for the requested marker construction:

```
Delivered:
REQ agent_a build.component(bind=MARKER_TERMINAL,q=1,item=oak_planks,site=terminal_marker)
Generated:
SELF resource.deliver(bind=DEPOT_TERMINAL,q=2,dst=terminal_chest,item=oak_planks)
REQ -
```

Although both outputs have valid syntax and name a public branch, request checking rejects them because their work contradicts the delivered request. The examples therefore illustrate a consistency requirement applied before world execution.

Information absence and grounded outcomes. Without a request, DeepSeek forms 795 commitments and selects the correct assignment in 385 of them, or 48.4% of formed cases. Its all-assigned success is 40.1% because another 165 trials stop during formation, consistent with an information upper bound that permits lower success. Across both backends and conditions, all 2,807 formed plans selecting the wrong goal fail the task, whereas all 4,684 matching plans succeed. This verifies the link from the admitted branch to the mutually exclusive terminal target.

S12.5.3 Model, Prompt, and Generation Settings. The request-transfer diagnostic called DeepSeek-V4.1-Flash through the service alias `deepseek-flash` on 2026-09-28, using the prompt and settings below. The API documentation citation records its consultation date of September 29 [15], distinct from the experiment's access date.

We used an Anthropic-compatible endpoint with `thinking.type` set to `disabled` and an 8,192-token output limit. The request also set `output_config.effort` to `high`, while temperature, top-p, seed, tools, and stop sequences were omitted. Each call supplied one user message and the diagnostic allowed four concurrent calls. Parsing removed an optional outer Markdown fence before applying the task contract, and unsuccessful generations remained in the assigned outcomes.

Each prompt provides four training examples, one from Active Order Branch, Build Site A/B, Chest/Crafting Table, and Deposit/Build. Sender and receiver calls use the examples for their own stage, beginning with the following instruction:

```
Four TRAIN syntax examples; solve only the final MODEL_INPUT.
```

After the examples, the prompt appends `CURRENT INSTANCE` and the agent's local input, followed by the task instruction:

```
Use only MODEL_INPUT. Generate self tasks and, when locally justified, one peer request. Return exactly two Short DSL lines:
     SELF ... and REQ .... No prose or semantic repair.
```

The local input identifies the agent and includes its task structure, observations, history, and delivered messages, while the sender's private choice appears only in the sender view. The accompanying data contain the complete examples and prompts. Local calls use their checkpoint prompts with deterministic decoding and a 128-token output cap, so the common interface is evaluated with each backend's stated generation settings.

S12.5.4 API Timing and Generation Settings.

API deployment timing. We also time the two-call API protocol on 64 role instances repeated three times, with sequential calls and no model warm-up. Among 192 trials, 189 form commitments with median TTC 1.52 s, mean 2.45 s, 95th percentile 2.70 s, and maximum 154.24 s. These measurements include the service, network, and retry time of the evaluated API interaction.

The diagnostic uses non-thinking generation with an 8,192-token cap, allowing the complete structured response to be returned under the prompt specified above.

S13 Structural Cues in Formation

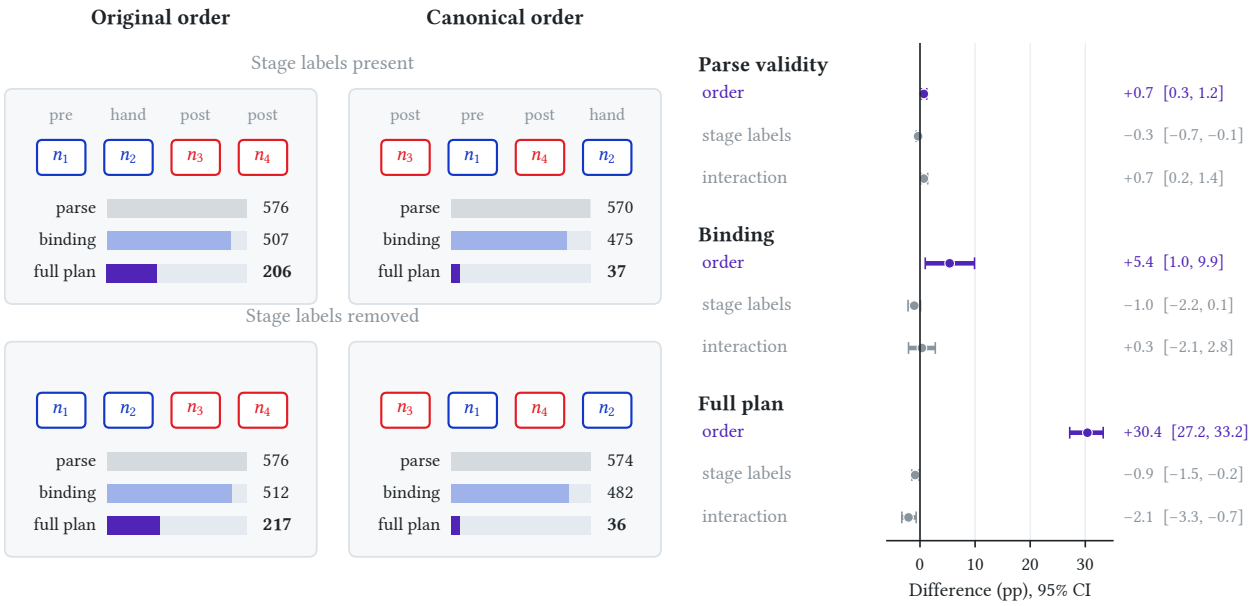
This section supports main Section 4 by examining how task-graph visibility and node order affect generated paths.

S13.1 Visibility Projections

Figure S6(c) compares Direct, controlled free-form, and a symbolic rule across 96 clusters. Full supplies typed nodes, stages, dependencies, and edges. Skeleton removes dependencies, whereas Node-only also removes stages and orders nodes independently of the answer. The symbolic rule uses a fixed path priority. Direct completes every familiar post-handoff case under Full and Skeleton, while both new transformation-order motifs have zero success.

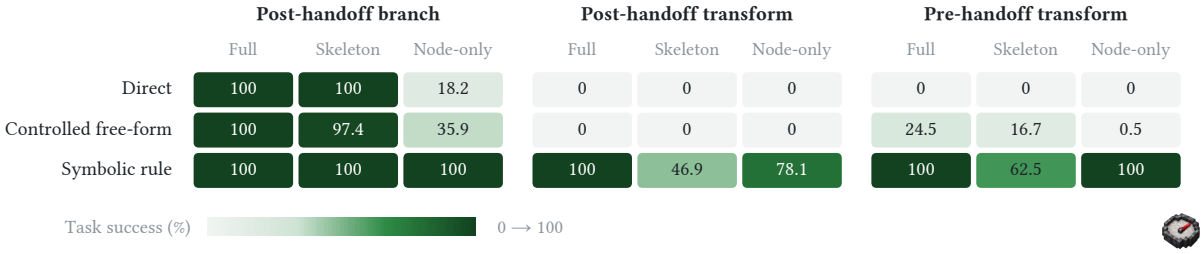
S13.2 Stage Labels and Node Order

We cross stage-label presence with original versus canonical order on the same nodes, fields, targets, and checkpoints. Each view contains 576 evaluations over 96 clusters, two roles, and three checkpoints (Figure S6(a,b)). Main effects average over the other factor, and interaction compares the order effects. We average roles and checkpoints within cluster and use 10,000 paired resamples within template–motif strata. The positive complete-plan effect belongs to node order, whereas stage labels have a small negative average effect.



(a) Presentations and funnels · n = 576 per arm

(b) Order, stage-label, and interaction effects · 95% CI



(c) Original projections · task success by method, view, and path motif

Figure S6. Task structure and presentation order. Panel (a) crosses stage labels with original or canonical node order and reports parsing, assignment, and complete-plan counts out of 576. Panel (b) gives paired factorial effects with 95% cluster intervals. Panel (c) compares the Full, Skeleton, and Node-only projections over 32 clusters per motif, using two roles and three checkpoints for learned methods and a seedless symbolic rule. Blue and red nodes indicate sender and receiver work.